Import the necessary libraries

In [1]:
```
import seaborn as sns
import matplotlib.pyplot as plt
import scipy.stats as st
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
%matplotlib inline
sns.set(style="whitegrid")
```

Import Dataset

In [2]:
```
heart_data = pd.read_csv(r'D:\fsds\projects\datasets\heart.csv')
```

In [3]:
```
heart_data.head()
```

Out[3]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2

Exploratory Data Analysis

In [4]:
```
heart_data.shape
```

Out[4]: (1025, 14)

In [5]:
```
heart_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       1025 non-null   int64  
 1   sex       1025 non-null   int64  
 2   cp        1025 non-null   int64  
 3   trestbps  1025 non-null   int64  
 4   chol      1025 non-null   int64  
 5   fbs       1025 non-null   int64  
 6   restecg   1025 non-null   int64  
 7   thalach   1025 non-null   int64  
 8   exang     1025 non-null   int64  
 9   oldpeak   1025 non-null   float64
 10  slope     1025 non-null   int64  
 11  ca        1025 non-null   int64  
 12  thal      1025 non-null   int64  
 13  target    1025 non-null   int64  
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

In [6]:
```
heart_data.dtypes
```

Out[6]:
```
age           int64
sex           int64
cp            int64
trestbps      int64
chol          int64
fbs           int64
restecg       int64
thalach       int64
exang         int64
oldpeak     float64
slope         int64
ca            int64
thal          int64
target        int64
dtype: object
```

In [7]:
```
heart_data.describe()
```

Out[7]:

	age	sex	cp	trestbps	chol	fbs	
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.00000	1025.000000	10
mean	54.434146	0.695610	0.942439	131.611707	246.00000	0.149268	
std	9.072290	0.460373	1.029641	17.516718	51.59251	0.356527	
min	29.000000	0.000000	0.000000	94.000000	126.00000	0.000000	
25%	48.000000	0.000000	0.000000	120.000000	211.00000	0.000000	
50%	56.000000	1.000000	1.000000	130.000000	240.00000	0.000000	
75%	61.000000	1.000000	2.000000	140.000000	275.00000	0.000000	
max	77.000000	1.000000	3.000000	200.000000	564.00000	1.000000	

In [8]:
```
heart_data.columns
```

Out[8]:
```
Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
       'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'],
      dtype='object')
```

Univariant Analysis

In [9]:
```
heart_data['target'].nunique()
```

Out[9]:
```
2
```

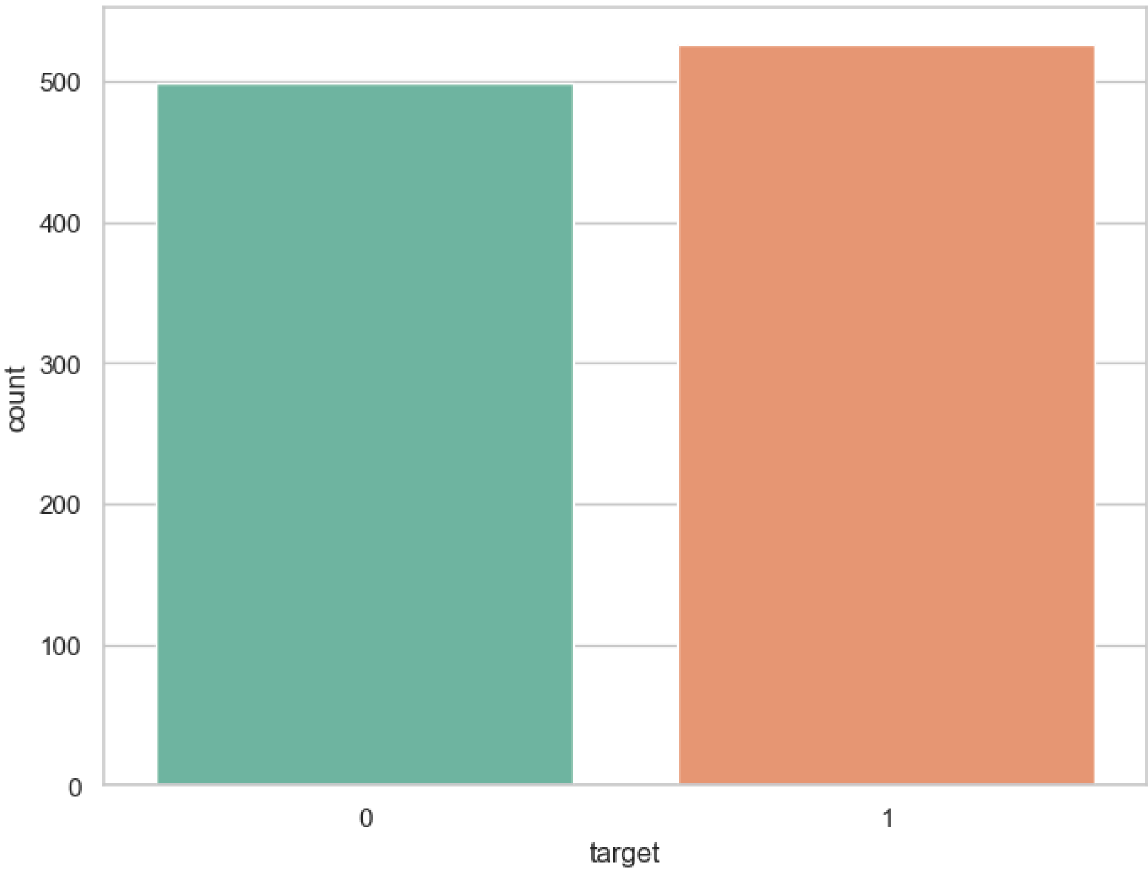
In [10]:
```
heart_data['target'].unique()
```

Out[10]:
```
array([0, 1], dtype=int64)
```

In [11]:
```
heart_data['target'].value_counts()
```

Out[11]:
```
target
1    526
0    499
Name: count, dtype: int64
```

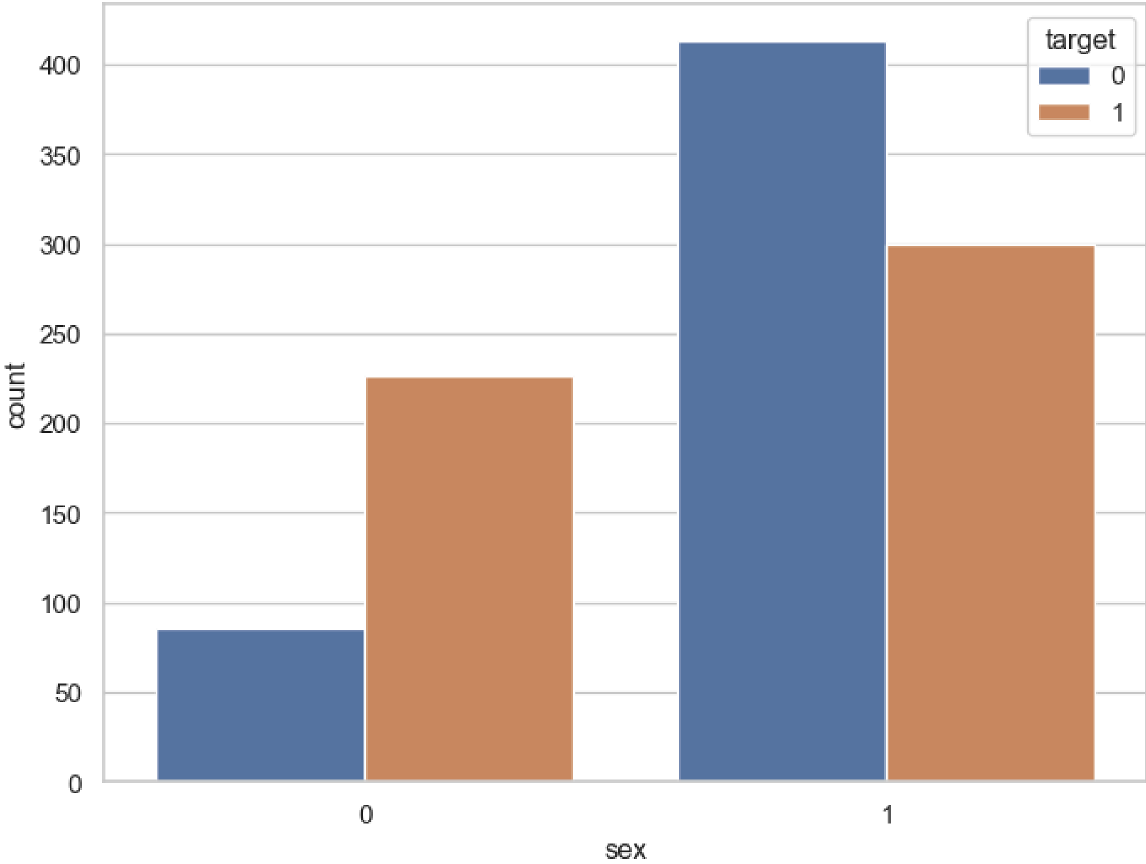
In [12]:
```
f, ax=plt.subplots(figsize=(8,6))
sx=sns.countplot(x='target', data=heart_data, palette='Set2')
plt.show()
```



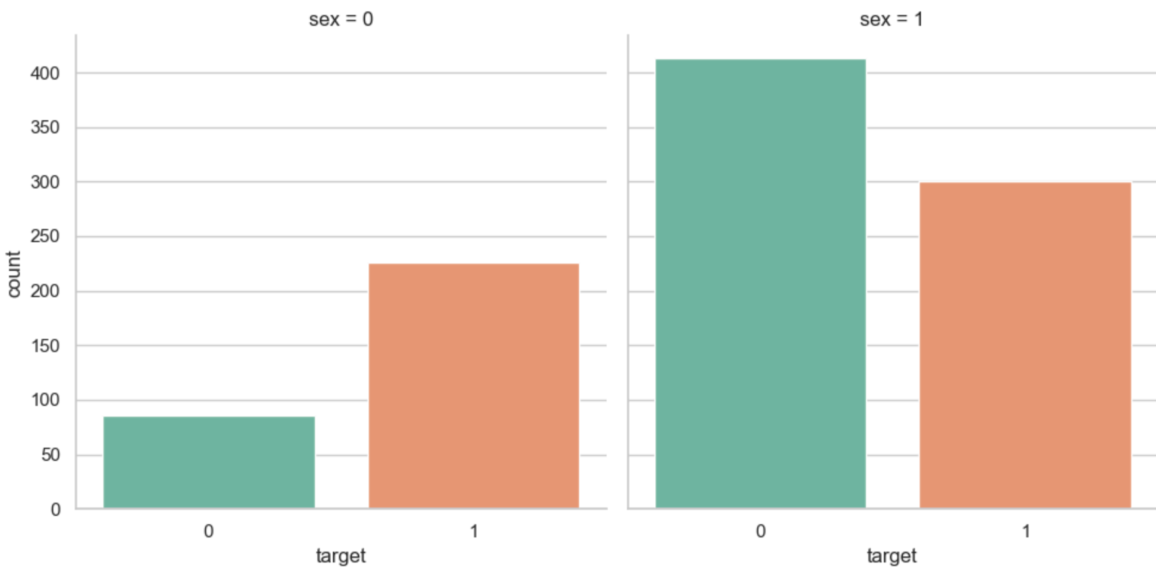
In [13]:
```
heart_data.groupby('sex')['target'].value_counts()
```

Out[13]:
```
sex  target
0    1         226
     0          86
1    0         413
     1         300
Name: count, dtype: int64
```

In [14]:
```
f, ax=plt.subplots(figsize=(8,6))
sx=sns.countplot(x='sex', data=heart_data, hue='target')
plt.show()
```

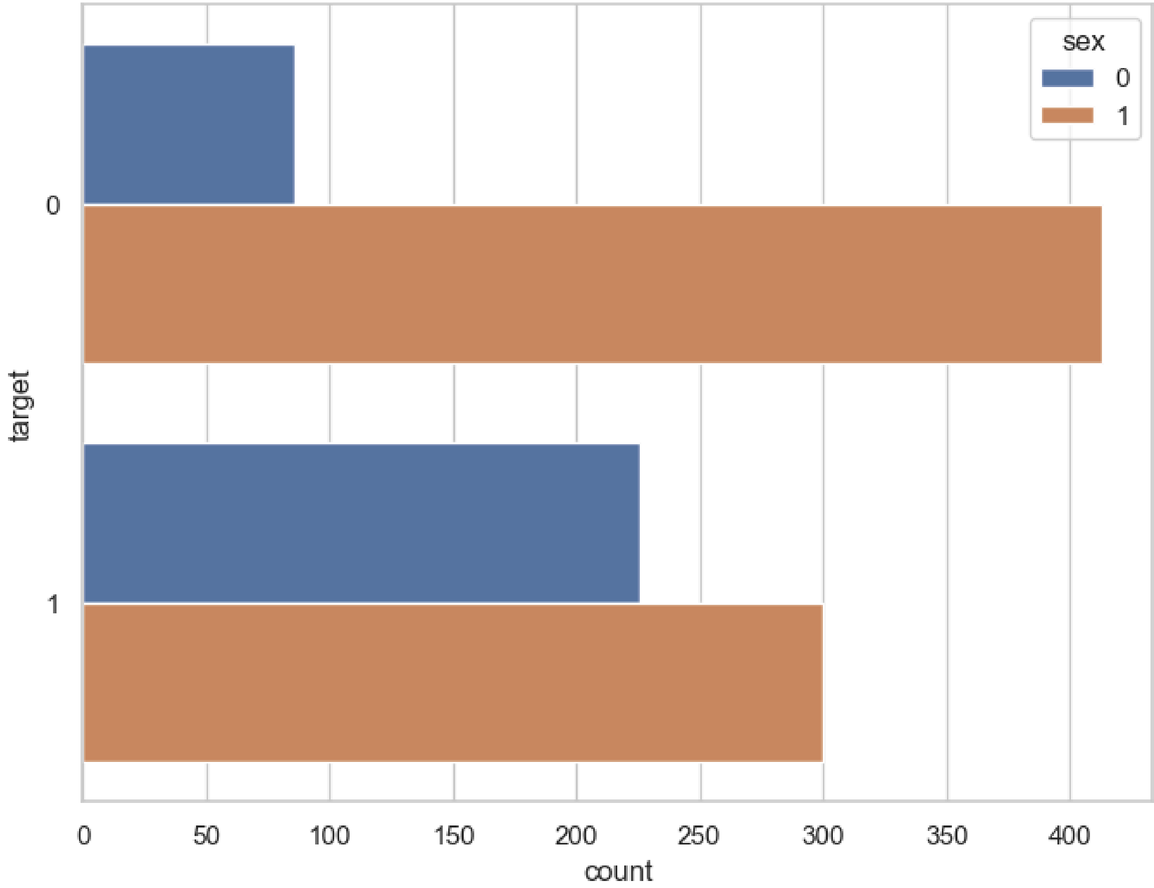


In [15]:
```
ax=sns.catplot(x='target',data=heart_data, col='sex', kind='count', height=5, as
```

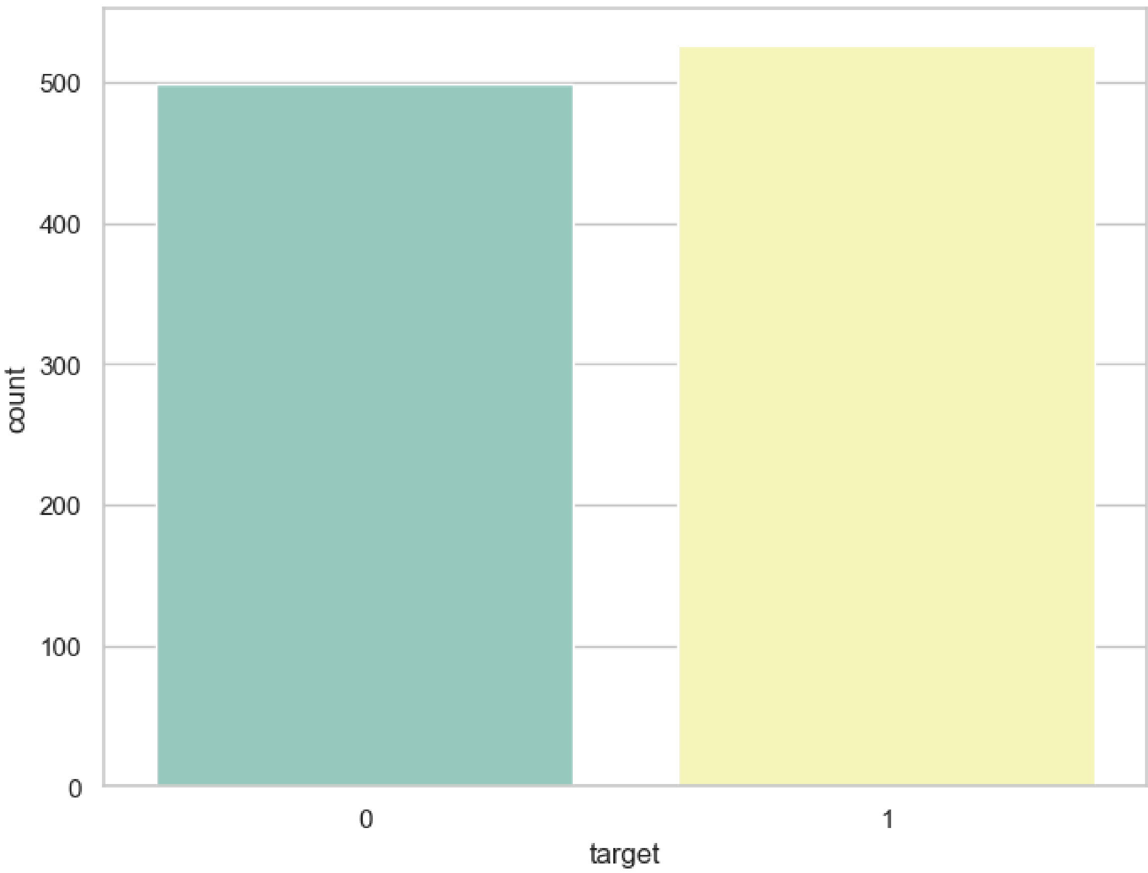


In [16]:
```
f,ax=plt.subplots(figsize=(8,6))
ax = sns.countplot(y='target', hue='sex', data=heart_data)
```

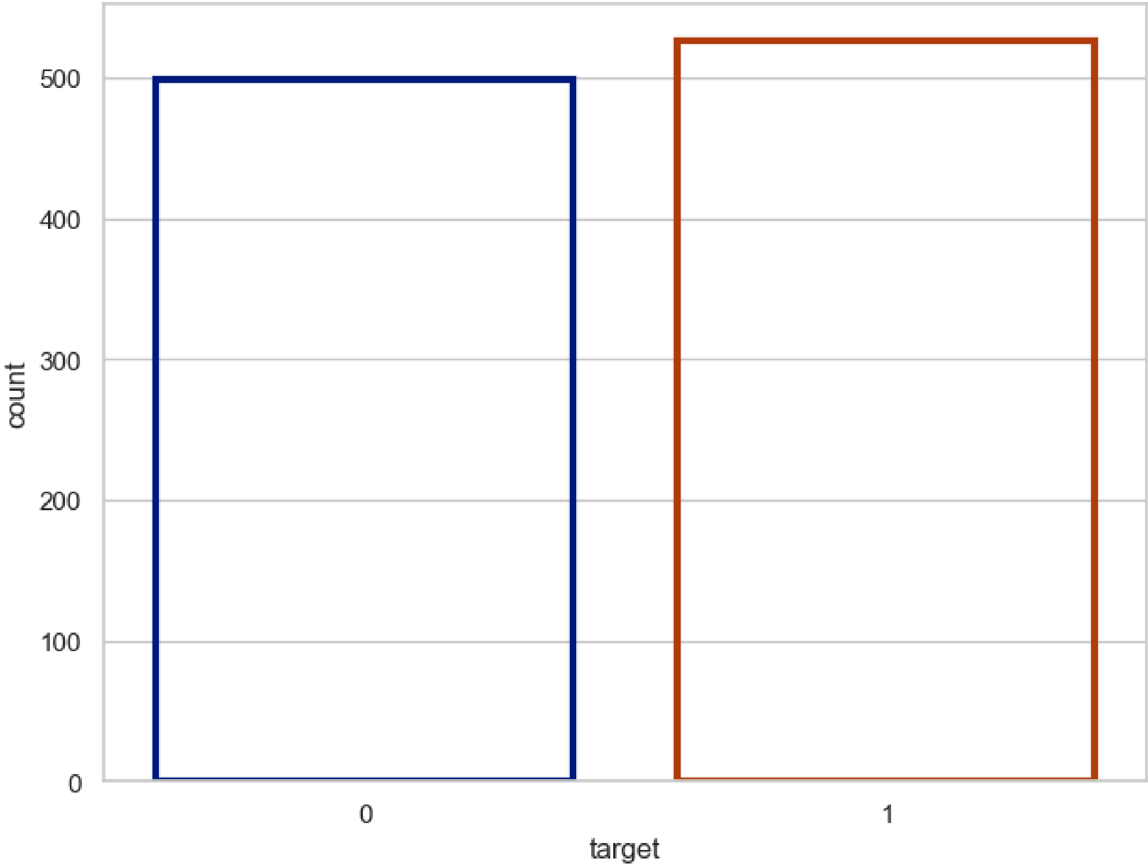
```
plt.show()
```



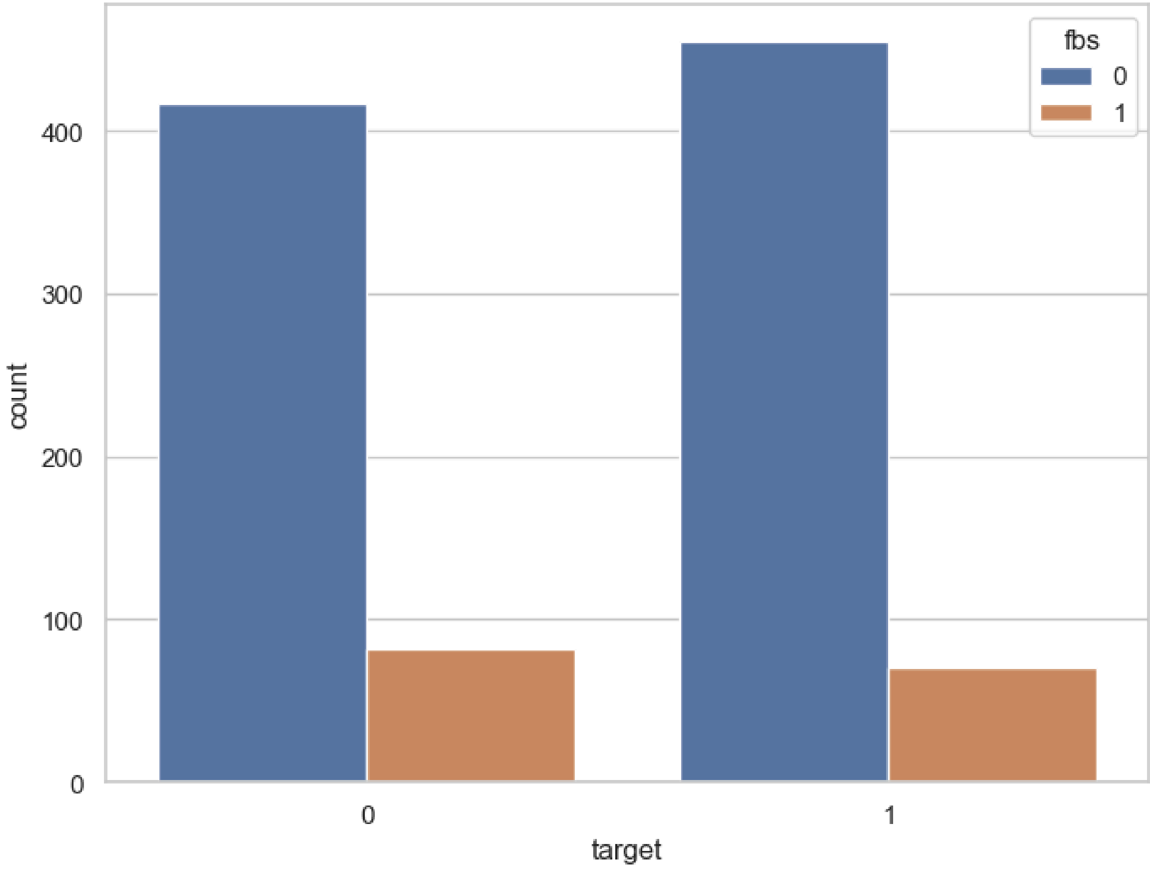
In [17]:
```
f,ax=plt.subplots(figsize=(8,6))
ax = sns.countplot(x='target', palette='Set3', data=heart_data)
plt.show()
```



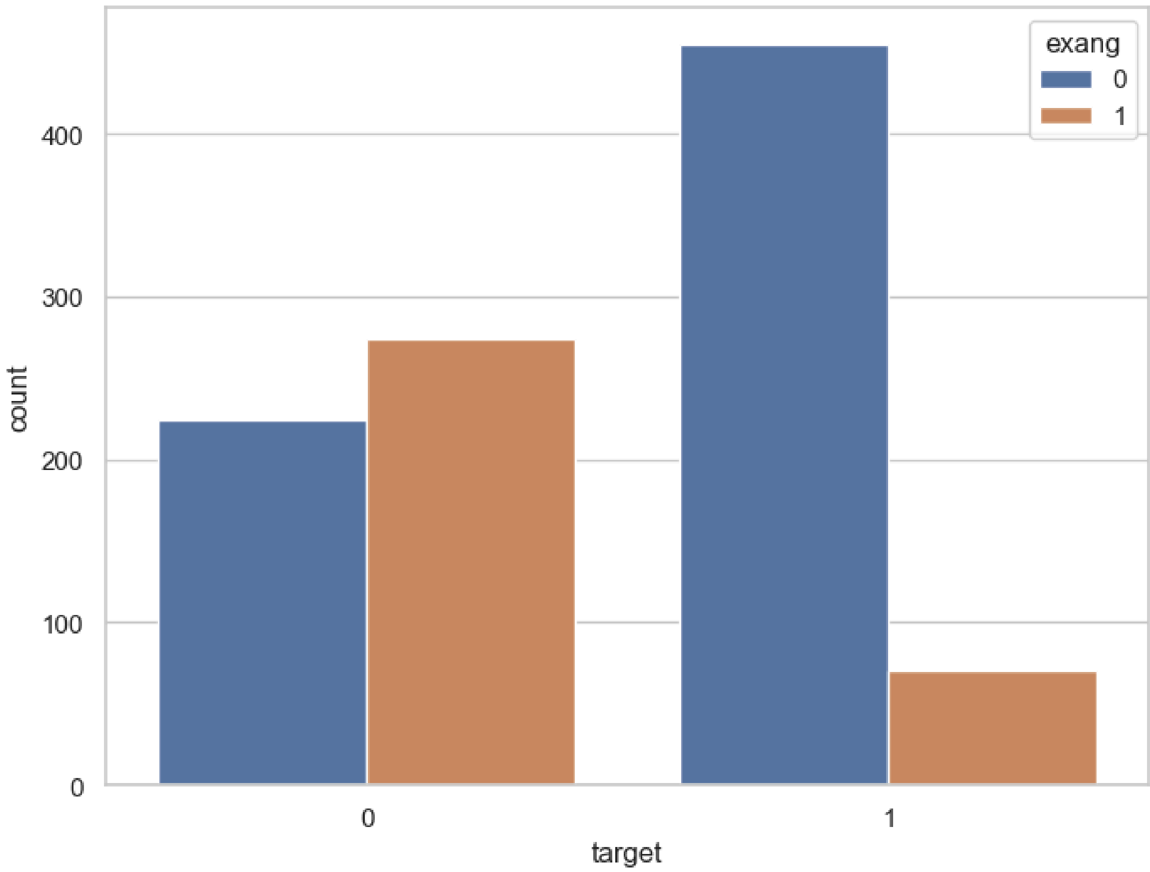
In [18]:
```
f,ax=plt.subplots(figsize=(8,6))
ax = sns.countplot(x='target', data=heart_data, facecolor=(0,0,0,0), linewidth=3
plt.show()
```



In [19]:
```
f,ax=plt.subplots(figsize=(8,6))
ax = sns.countplot(x='target', hue='fbs', data=heart_data)
plt.show()
```



In [20]:
```
f,ax=plt.subplots(figsize=(8,6))
ax = sns.countplot(x='target', hue='exang', data=heart_data)
plt.show()
```



In [21]:
```
heart_data['target'].value_counts()
```

Out[21]:
```
target
1    526
0    499
Name: count, dtype: int64
```

Bivariant Analysis

In [22]:
```
correlation = heart_data.corr() #corr() is used to find the correlation between
```

In [23]:
```
correlation['target'].sort_values(ascending=False)
```

Out[23]:
```
target      1.000000
cp          0.434854
thalach     0.422895
slope       0.345512
restecg     0.134468
fbs        -0.041164
chol       -0.099966
trestbps   -0.138772
age        -0.229324
sex        -0.279501
thal       -0.337838
ca         -0.382085
exang      -0.438029
oldpeak    -0.438441
Name: target, dtype: float64
```

In [24]:
```
df=heart_data
```

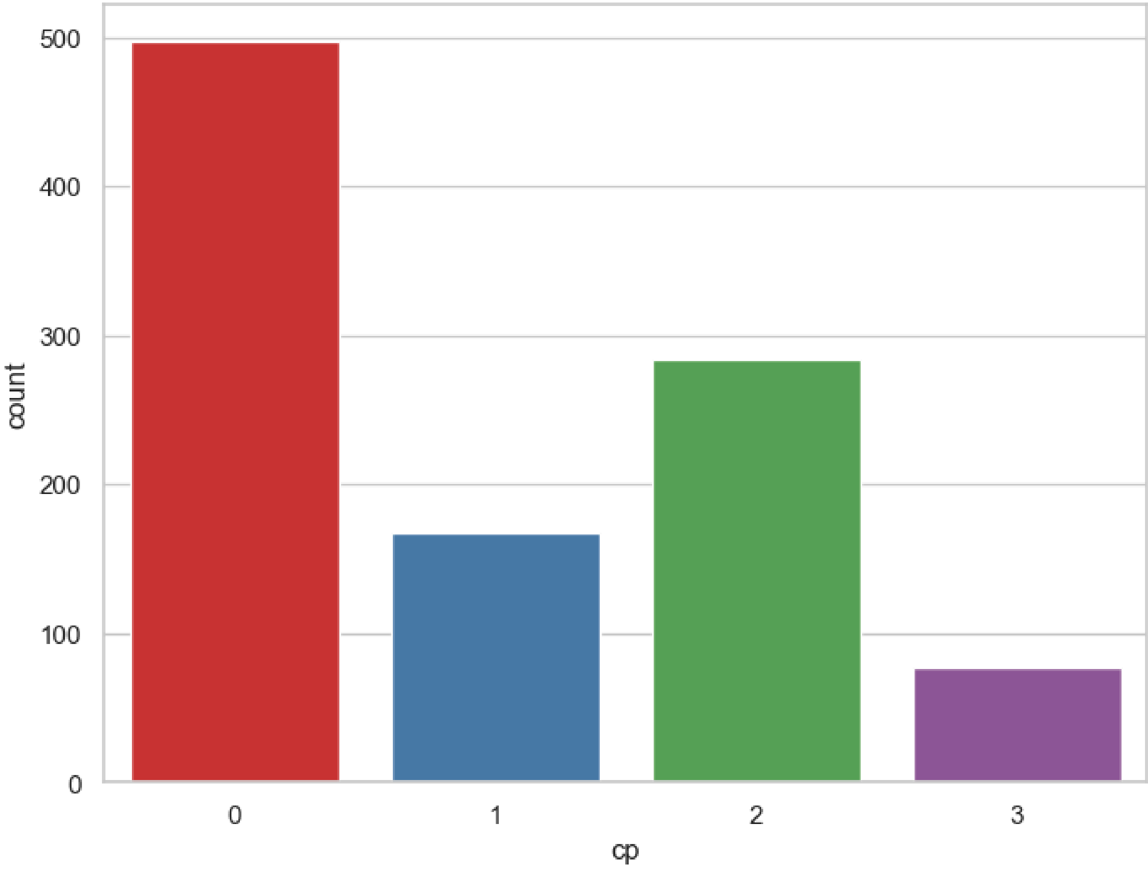
In [25]:
```
df['cp'].nunique
```

Out[25]:
```
<bound method IndexOpsMixin.nunique of 0       0
1       0
2       0
3       0
4       0
       ..
1020    1
1021    0
1022    0
1023    0
1024    0
Name: cp, Length: 1025, dtype: int64>
```

In [26]:
```
df['cp'].value_counts()
```

Out[26]:
```
cp
0    497
2    284
1    167
3     77
Name: count, dtype: int64
```

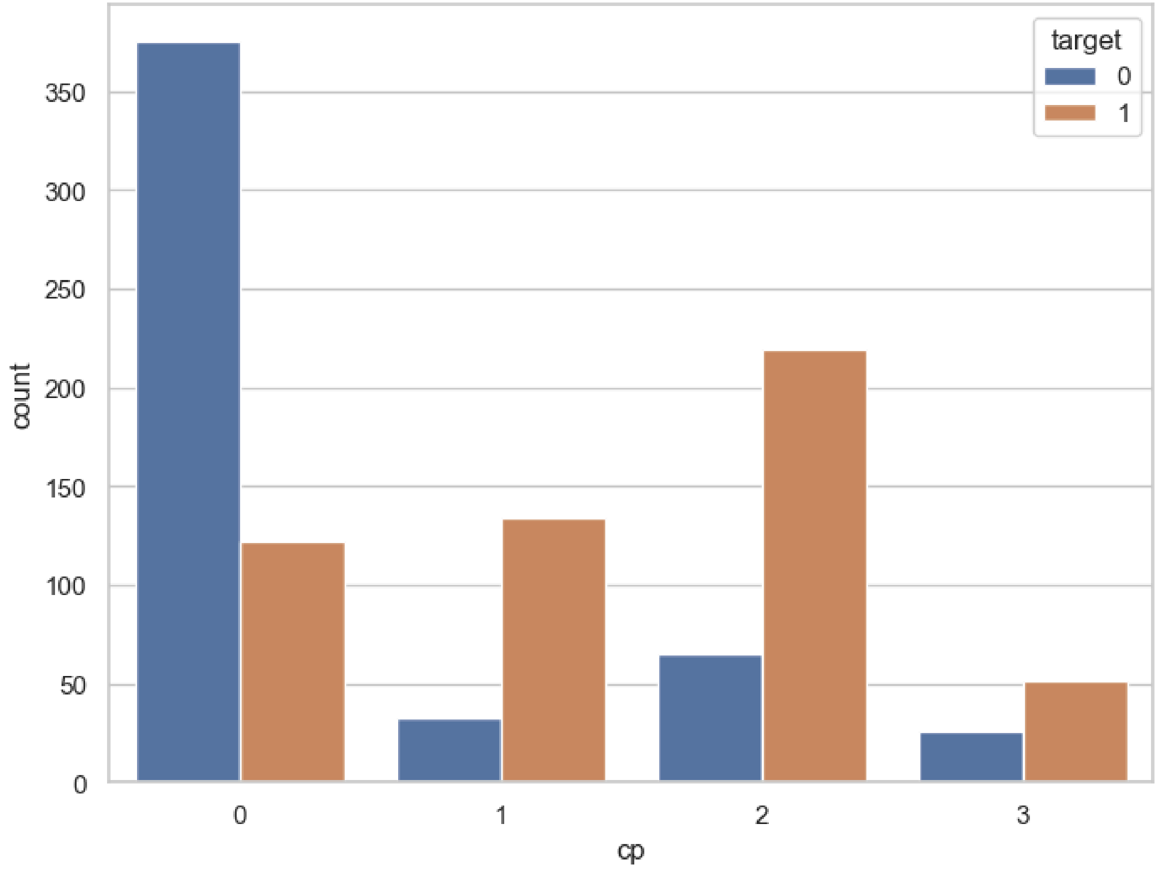
In [27]:
```
f,ax=plt.subplots(figsize=(8,6))
ax=sns.countplot(x='cp', data=df, palette='Set1')
plt.show()
```



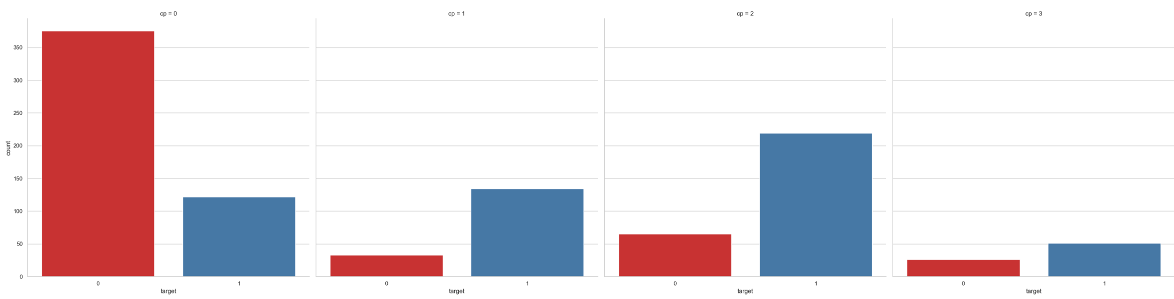
In [28]:
```
df.groupby('cp')['target'].value_counts()
```

Out[28]:
```
cp  target
0   0         375
    1         122
1   1         134
    0          33
2   1         219
    0          65
3   1          51
    0          26
Name: count, dtype: int64
```

In [29]:
```
f, ax = plt.subplots(figsize=(8, 6))
ax = sns.countplot(x="cp", hue="target", data=df)
plt.show()
```



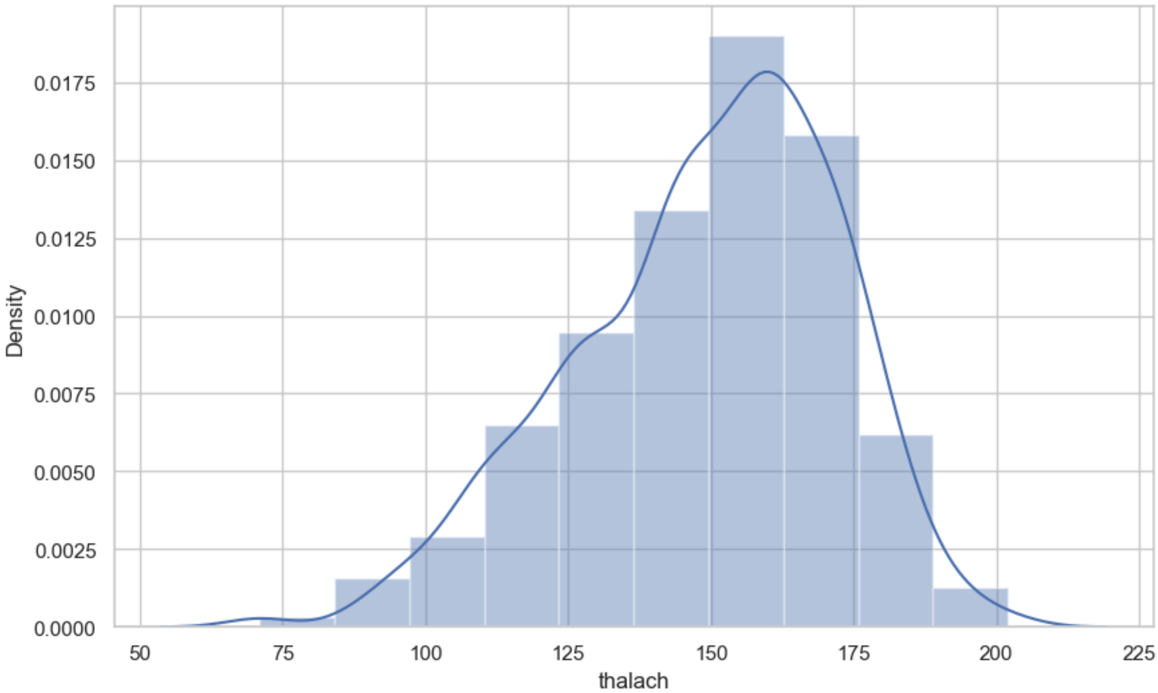
In [30]:
```
ax = sns.catplot(x="target", col="cp", data=df, kind="count", height=8, aspect=1
```



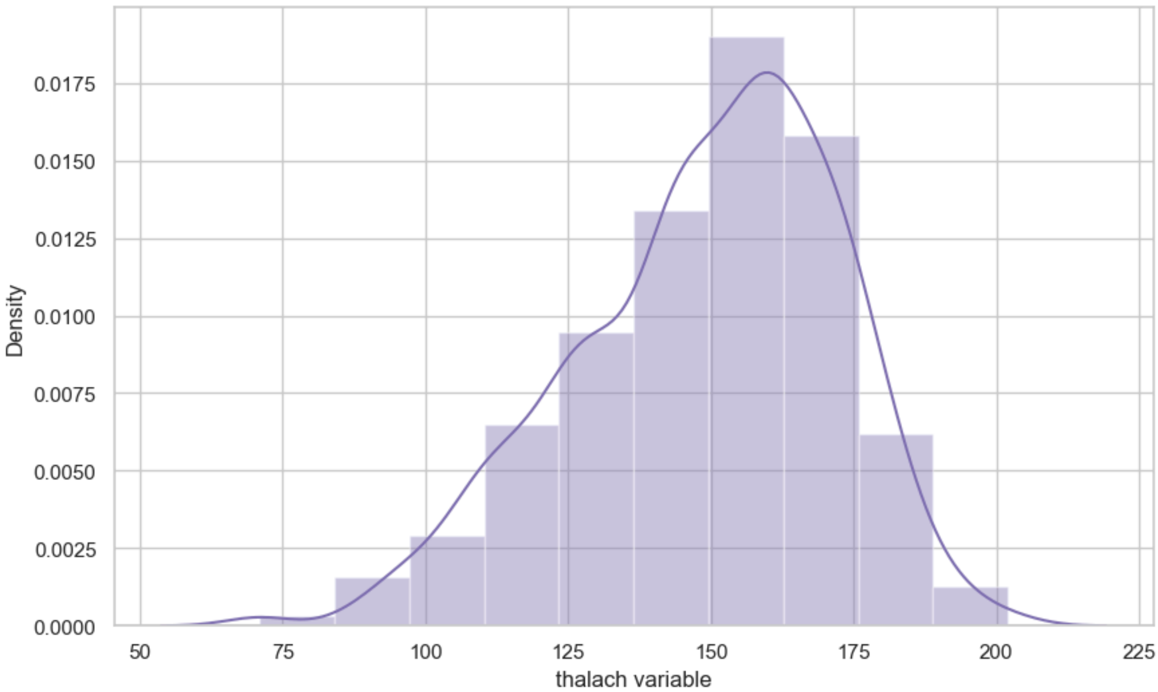
In [31]:
```
df['thalach'].nunique()
```

Out[31]: 91

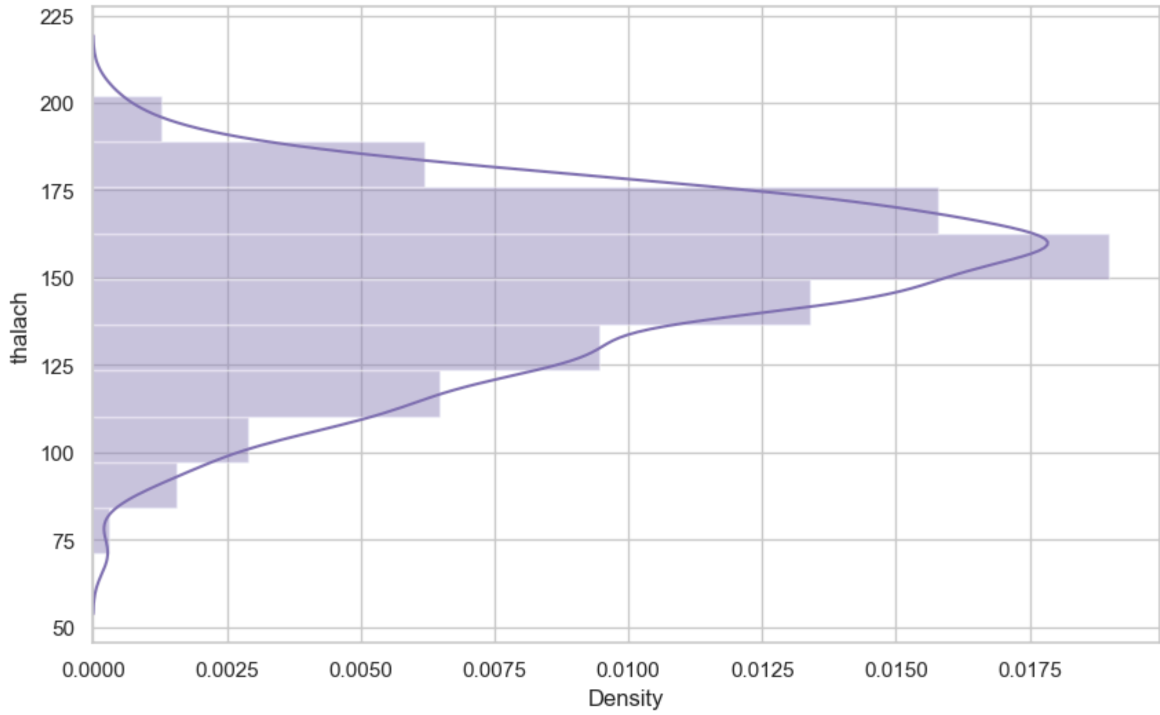
In [32]:
```
f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
ax = sns.distplot(x, bins=10)
plt.show()
```



In [33]:
```
f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
x = pd.Series(x, name="thalach variable")
ax = sns.distplot(x, bins=10, color='m')
plt.show()
```



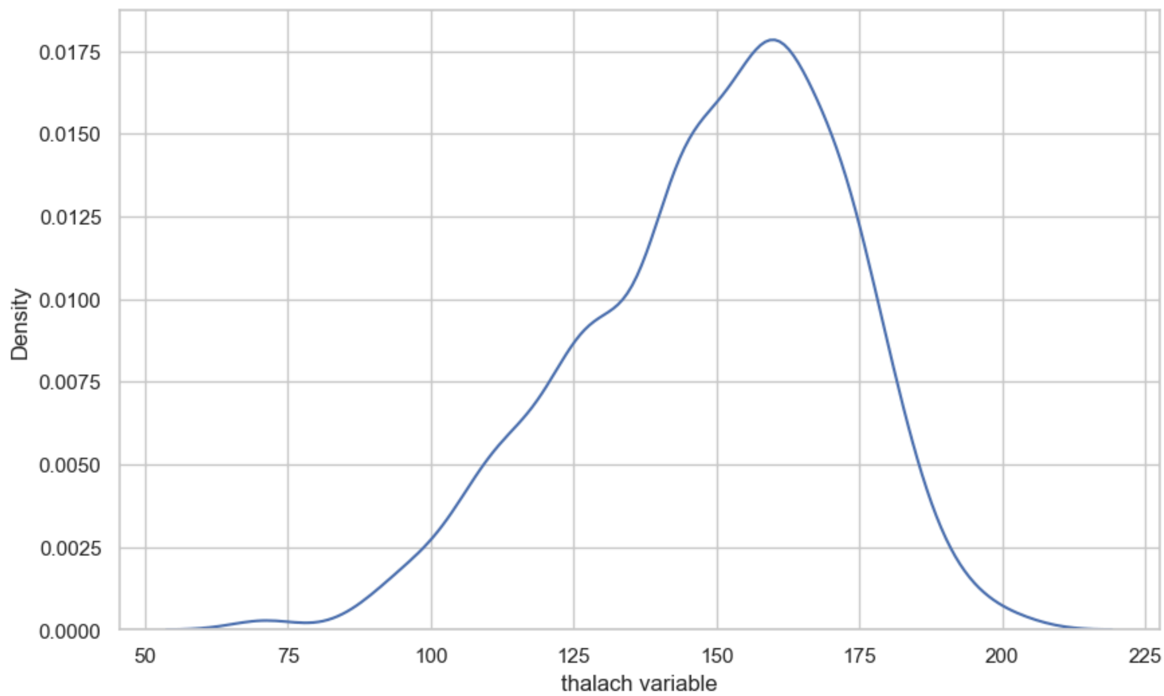
In [34]:
```
f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
ax = sns.distplot(x, bins=10, vertical=True, color='m')
plt.show()
```



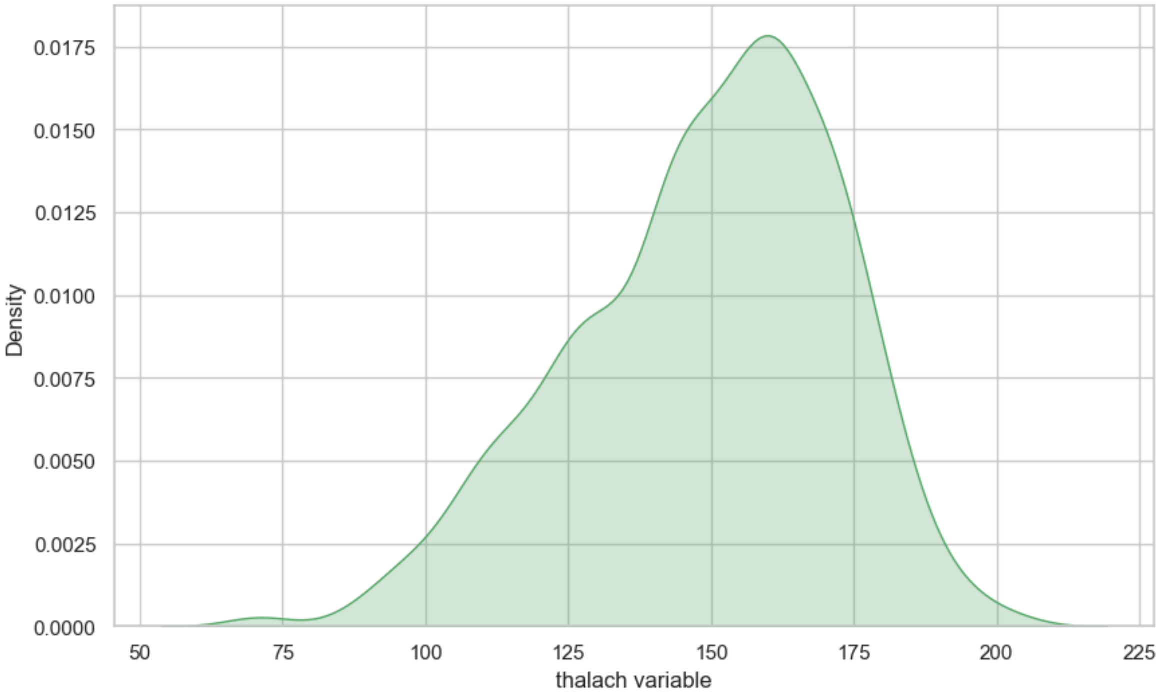
Seaborn Kernel Density Estimation (KDE) Plot

- The kernel density estimate (KDE) plot is a useful tool for plotting the shape of a distribution.
- The KDE plot plots the density of observations on one axis with height along the other axis.

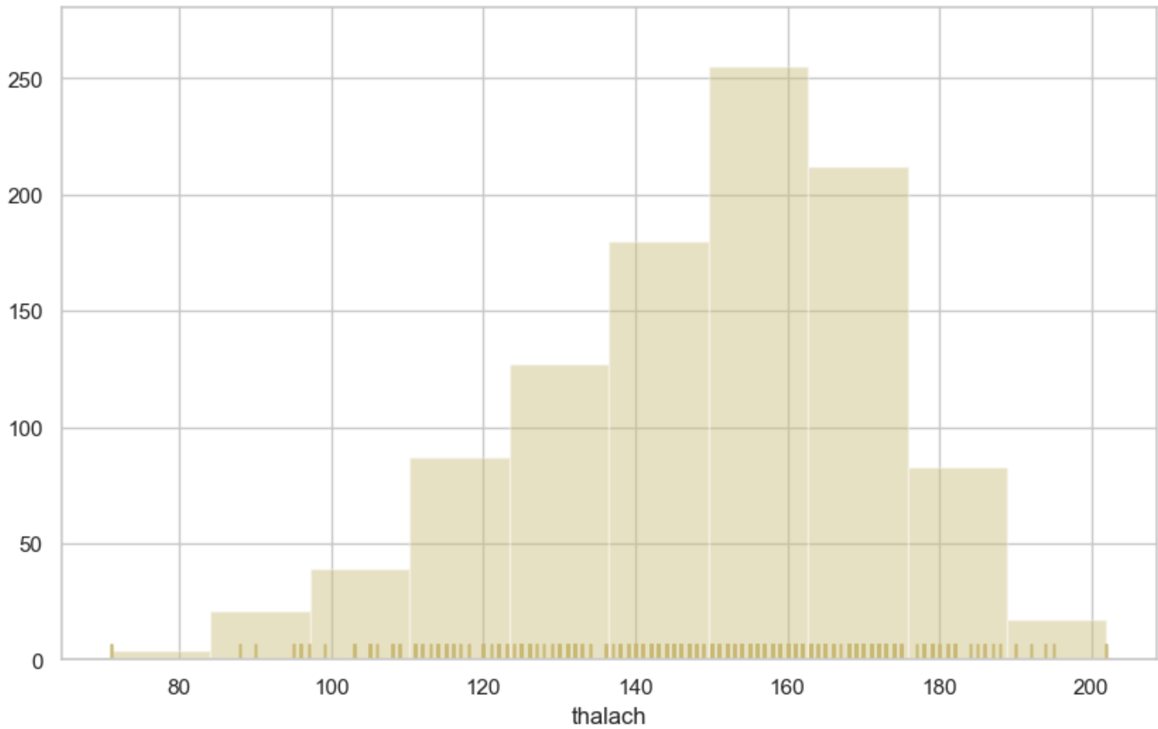
In [35]:
```
f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
x = pd.Series(x, name="thalach variable")
ax = sns.kdeplot(x)
plt.show()
```



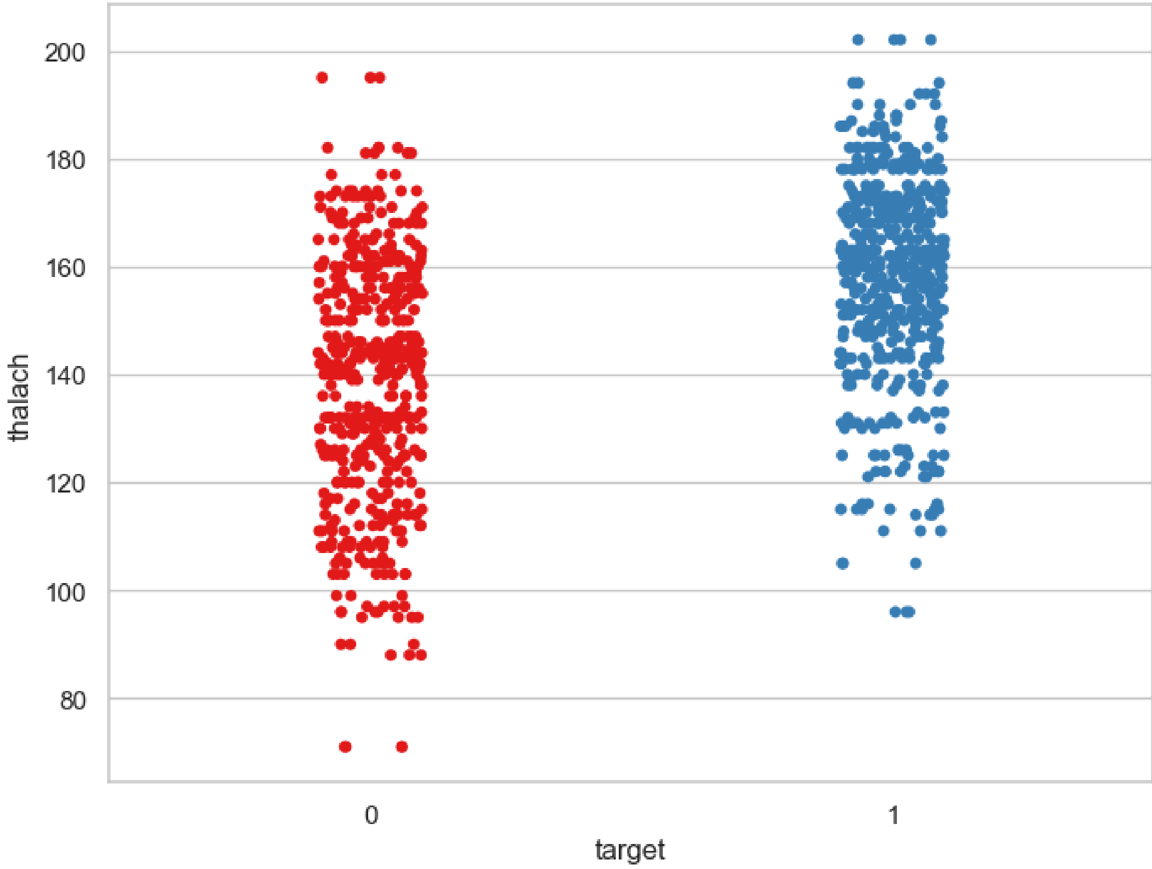
In [36]:
```
f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
x = pd.Series(x, name="thalach variable")
ax = sns.kdeplot(x, shade=True, color='g')
plt.show()
```



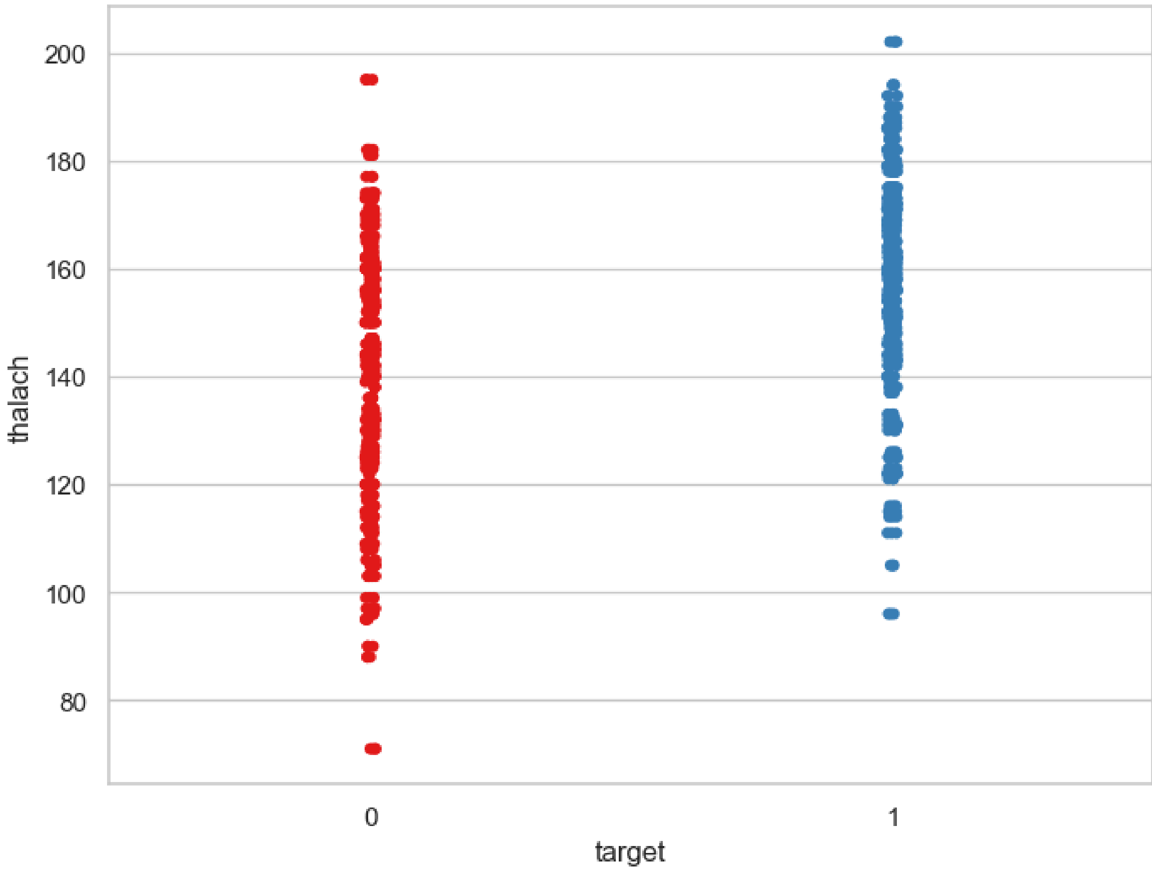
In [37]:
```
f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
ax = sns.distplot(x, kde=False, rug=True, bins=10, color='y')
plt.show()
```



In [38]:
```
f, ax = plt.subplots(figsize=(8, 6))
sns.stripplot(x="target", y="thalach", data=df, palette="Set1")
plt.show()
```

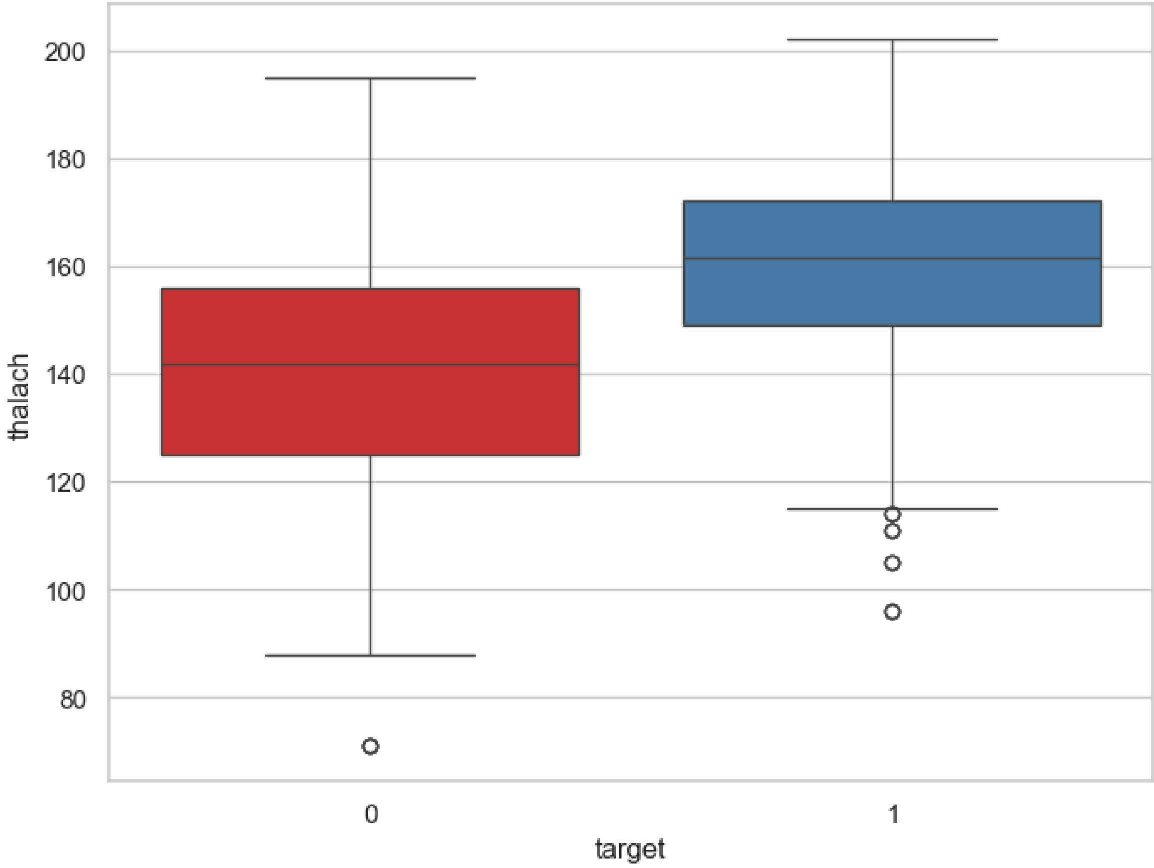


In [39]:
```
f, ax = plt.subplots(figsize=(8, 6))
sns.stripplot(x="target", y="thalach", data=df, jitter = 0.01, palette='Set1')
plt.show()
```



In [40]:
```
f, ax = plt.subplots(figsize=(8, 6))
sns.boxplot(x="target", y="thalach", data=df, palette="Set1")
```

```
plt.show()
```

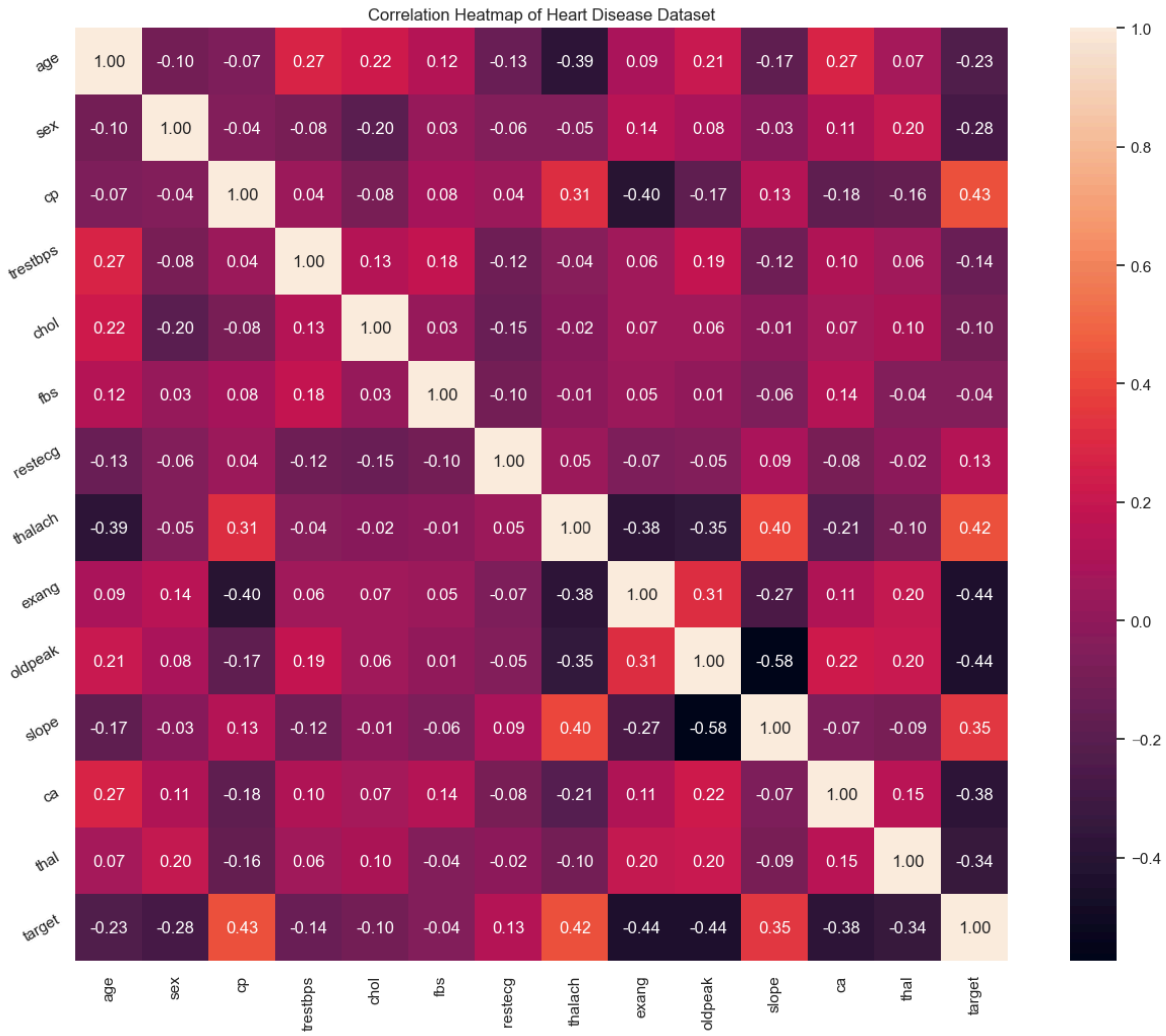


Multi Variant Analysis

Main objective is to find out relations between variables in the dataset

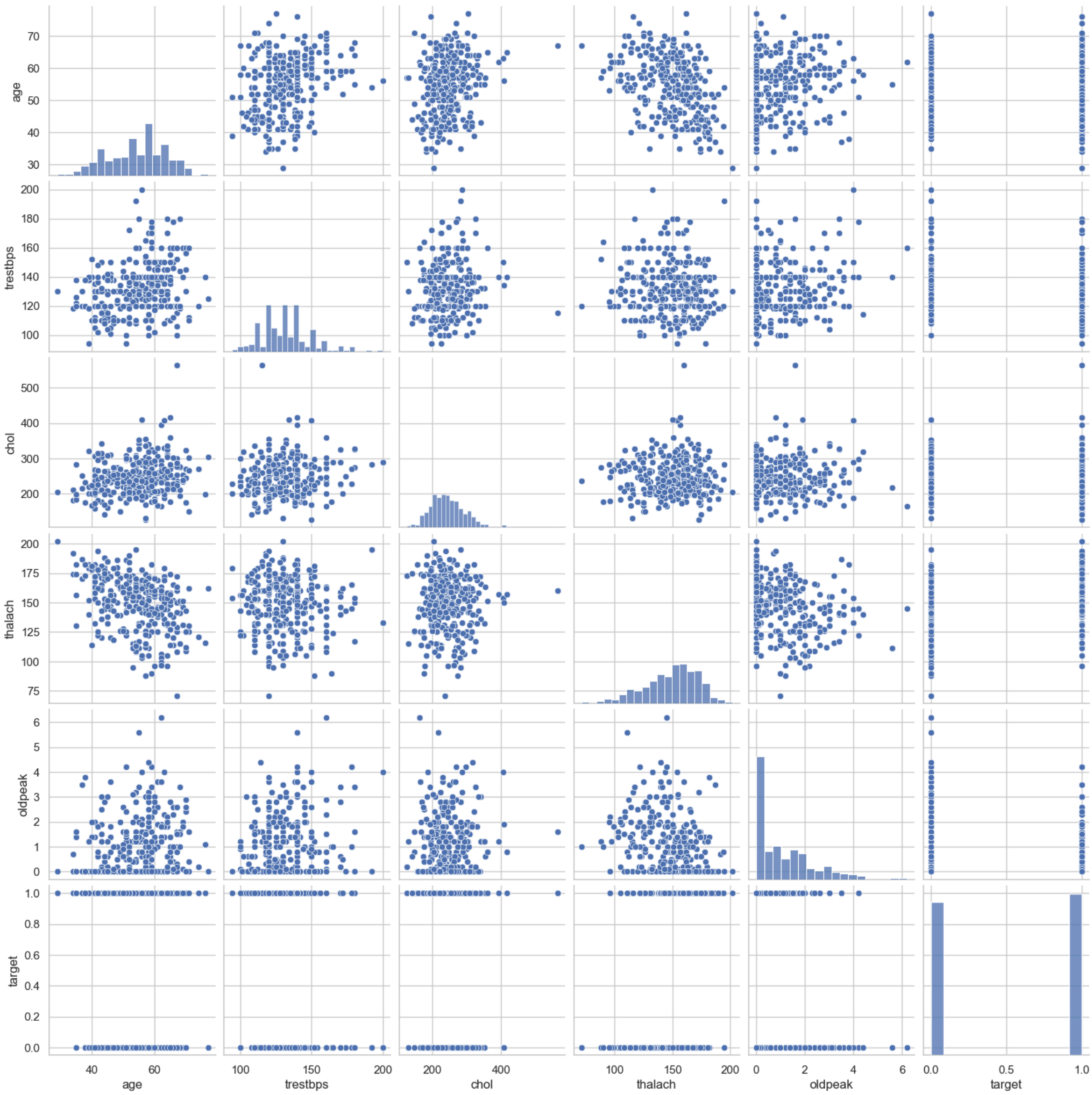
Heatmap

In [41]:
```
plt.figure(figsize=(16,12))
plt.title('Correlation Heatmap of Heart Disease Dataset')
a = sns.heatmap(correlation, square=True, annot=True, fmt='.2f', linecolor='whit
a.set_xticklabels(a.get_xticklabels(), rotation=90)
a.set_yticklabels(a.get_yticklabels(), rotation=30)
plt.show()
```



Pair Plot

In [42]:
```
num_var = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak', 'target' ]
sns.pairplot(df[num_var], kind='scatter', diag_kind='hist')
plt.show()
```



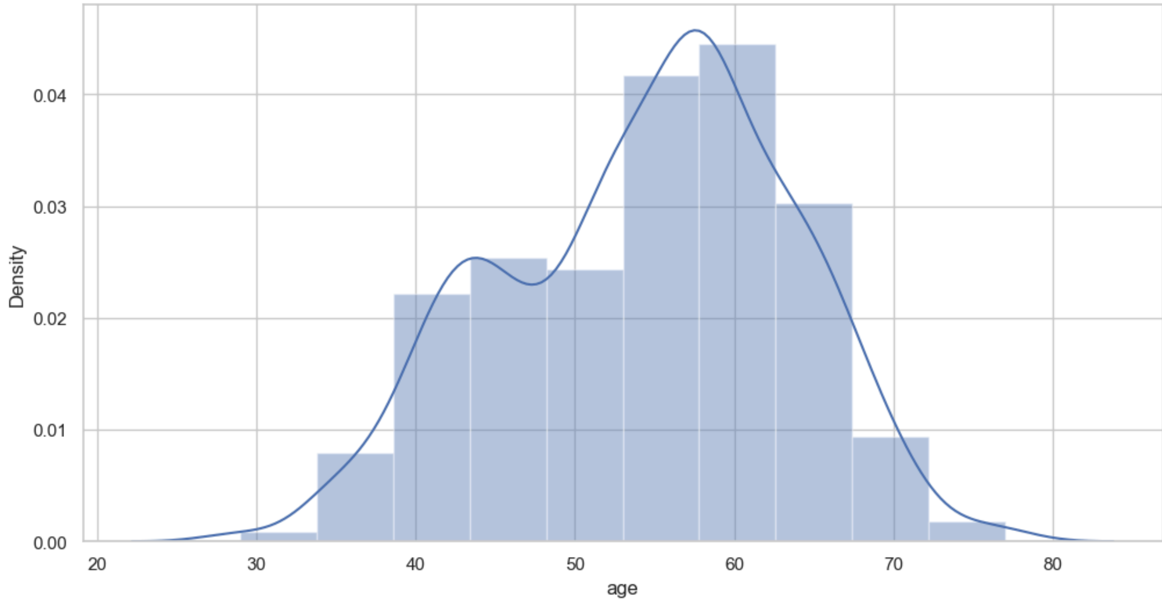
In [43]:
```python
df['age'].nunique()
```

Out[43]:
```
41
```

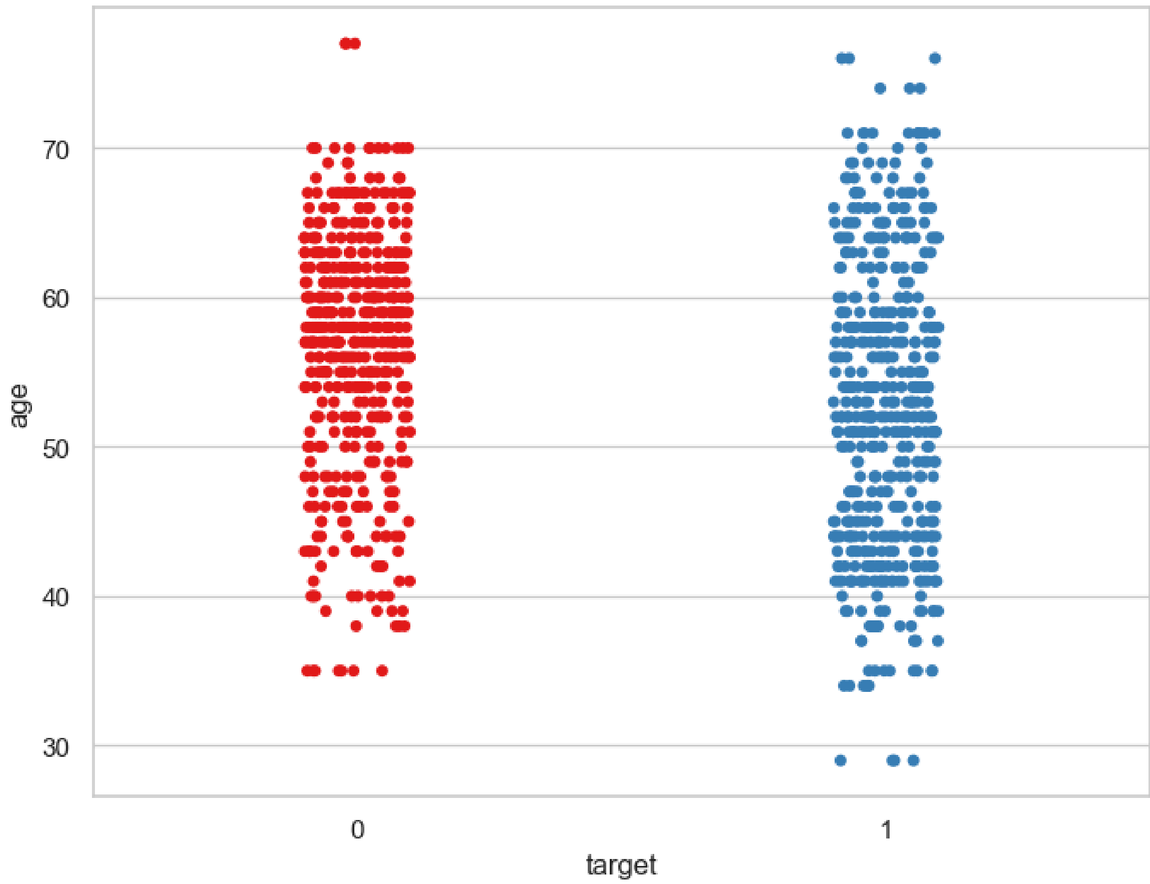
In [44]:
```python
df['age'].describe()
```

Out[44]:
```
count    1025.000000
mean       54.434146
std         9.072290
min        29.000000
25%        48.000000
50%        56.000000
75%        61.000000
max        77.000000
Name: age, dtype: float64
```

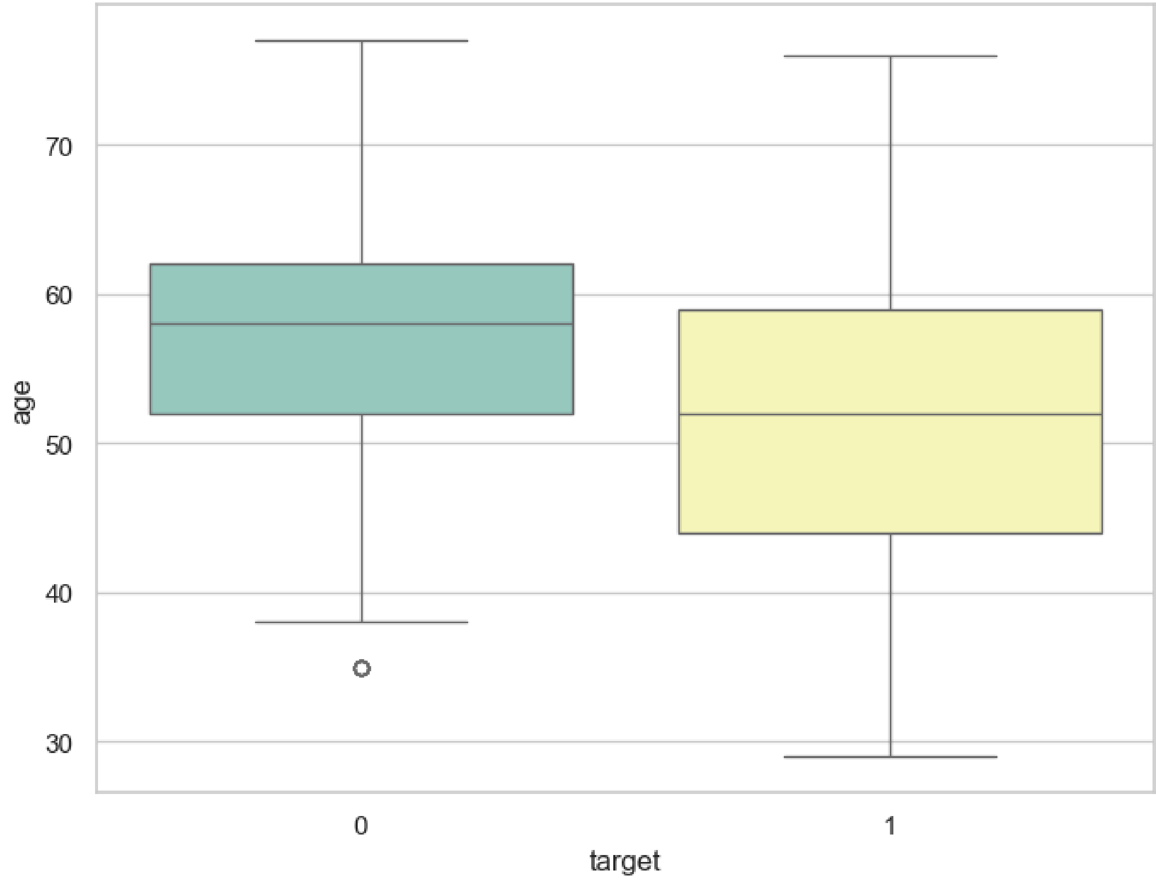
In [45]:
```python
f, ax = plt.subplots(figsize=(12,6))
x = df['age']
ax = sns.distplot(x, bins=10)
plt.show()
```



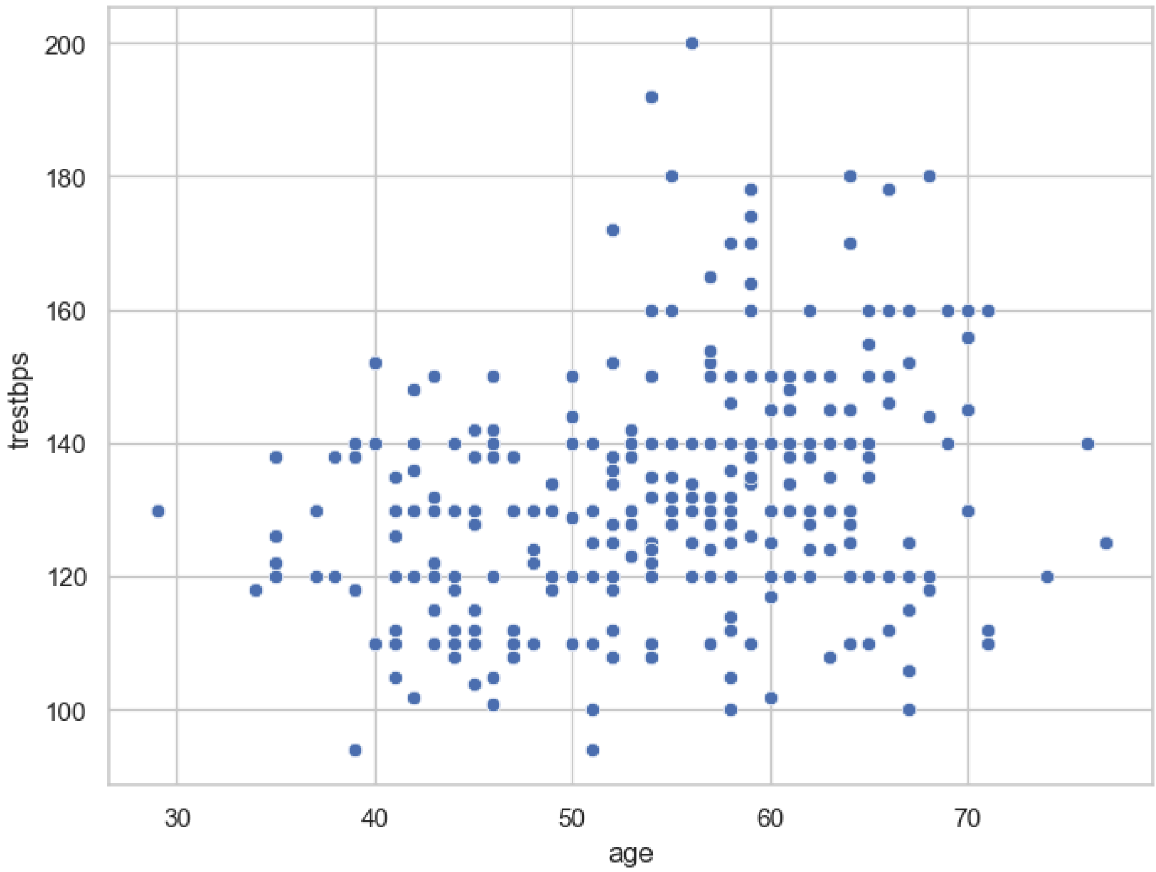
In [46]:
```
f, ax = plt.subplots(figsize=(8, 6))
sns.stripplot(x="target", y="age", data=df, palette="Set1")
plt.show()
```



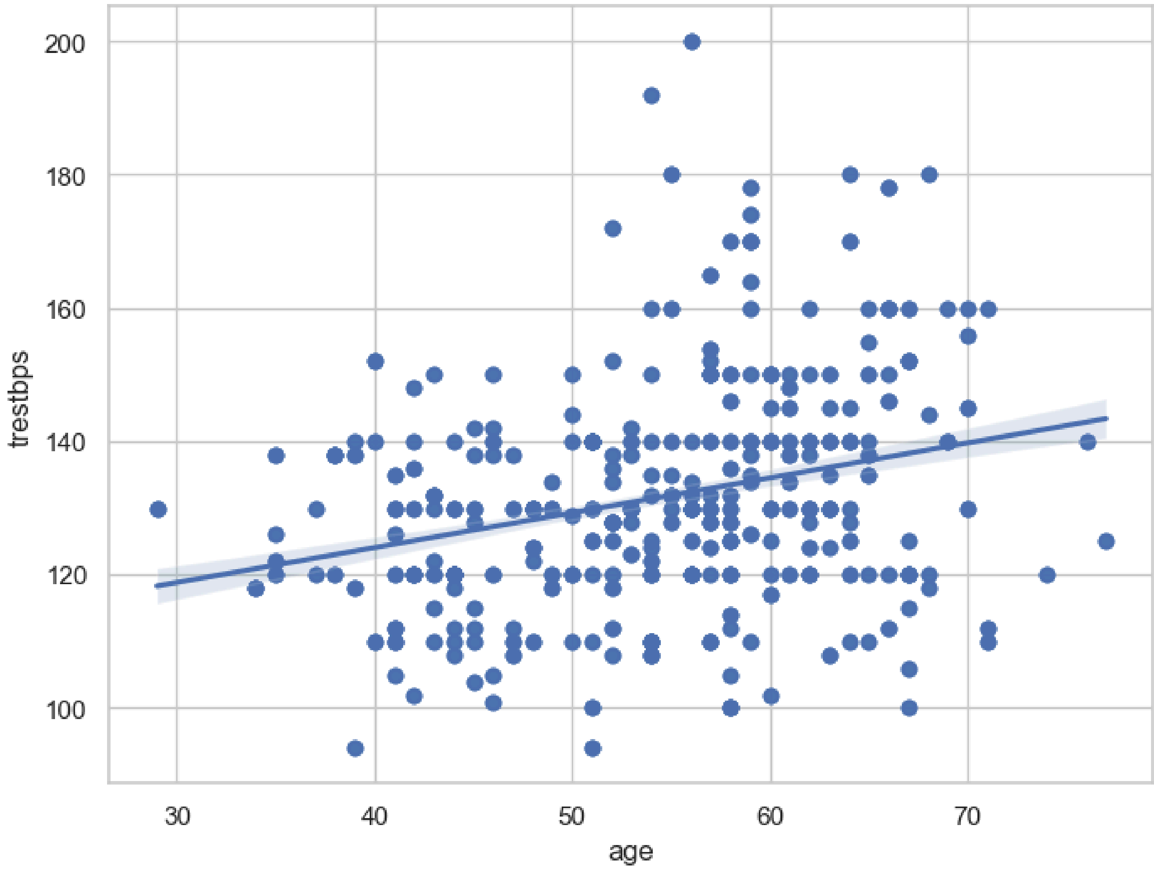
In [47]:
```
f, ax = plt.subplots(figsize=(8, 6))
sns.boxplot(x="target", y="age", data=df, palette="Set3")
plt.show()
```



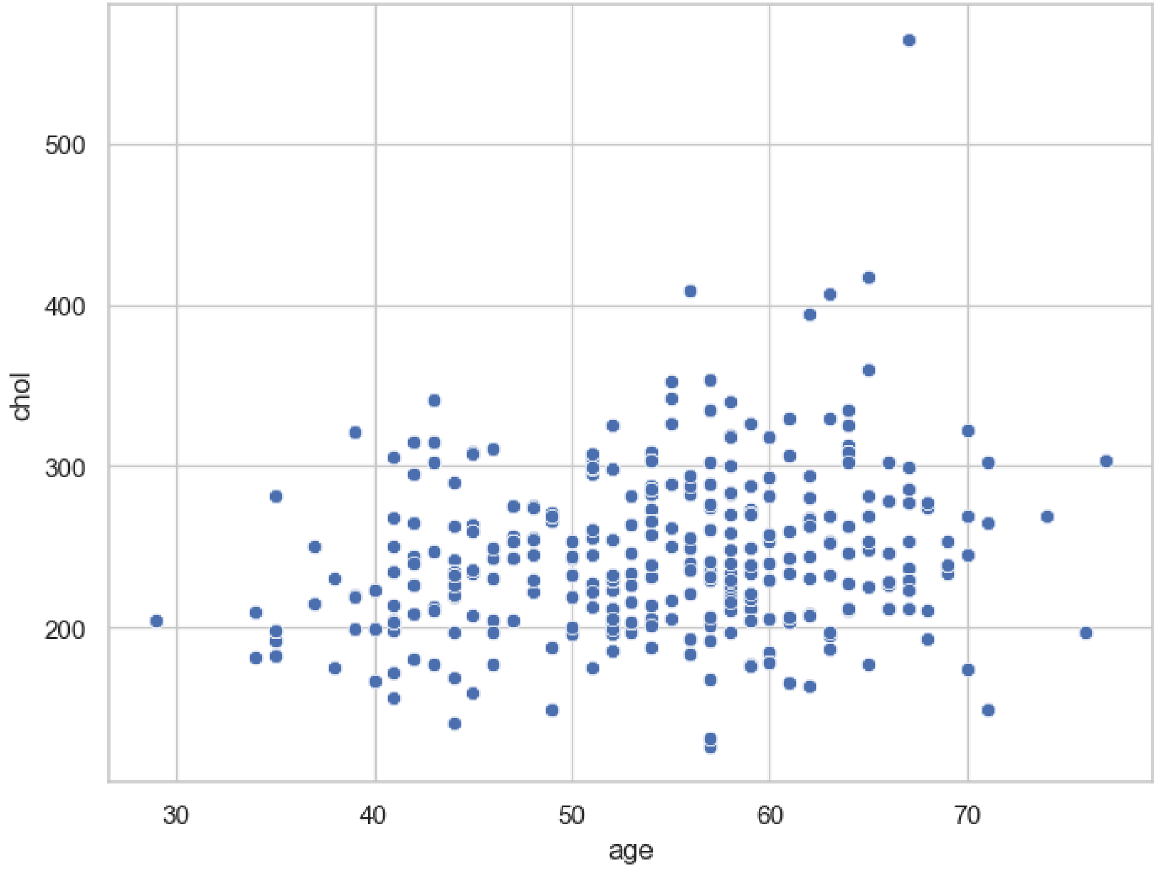
In [48]:
```
f, ax = plt.subplots(figsize=(8, 6))
ax = sns.scatterplot(x="age", y="trestbps", data=df)
plt.show()
```



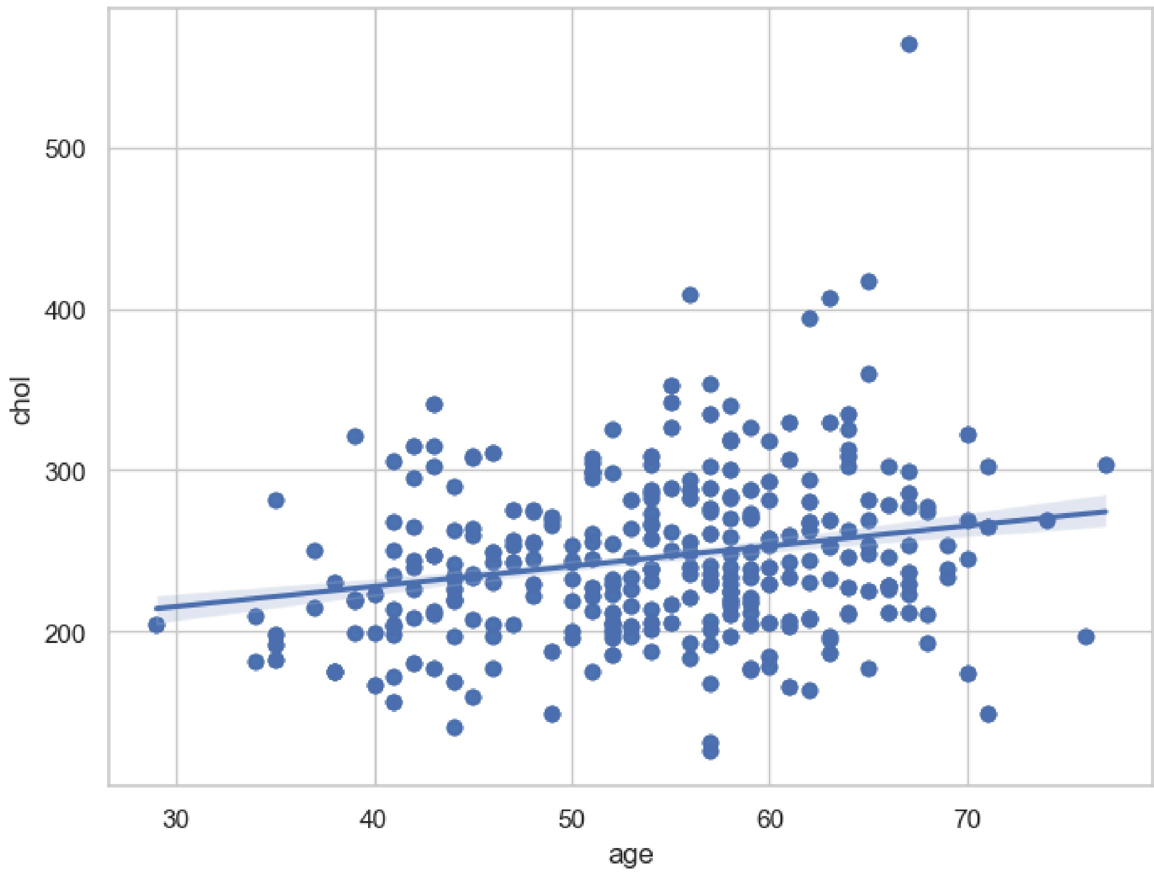
In [49]:
```
f, ax = plt.subplots(figsize=(8, 6))
ax = sns.regplot(x="age", y="trestbps", data=df)
plt.show()
```



In [50]:
```
f, ax = plt.subplots(figsize=(8, 6))
ax = sns.scatterplot(x="age", y="chol", data=df)
plt.show()
```

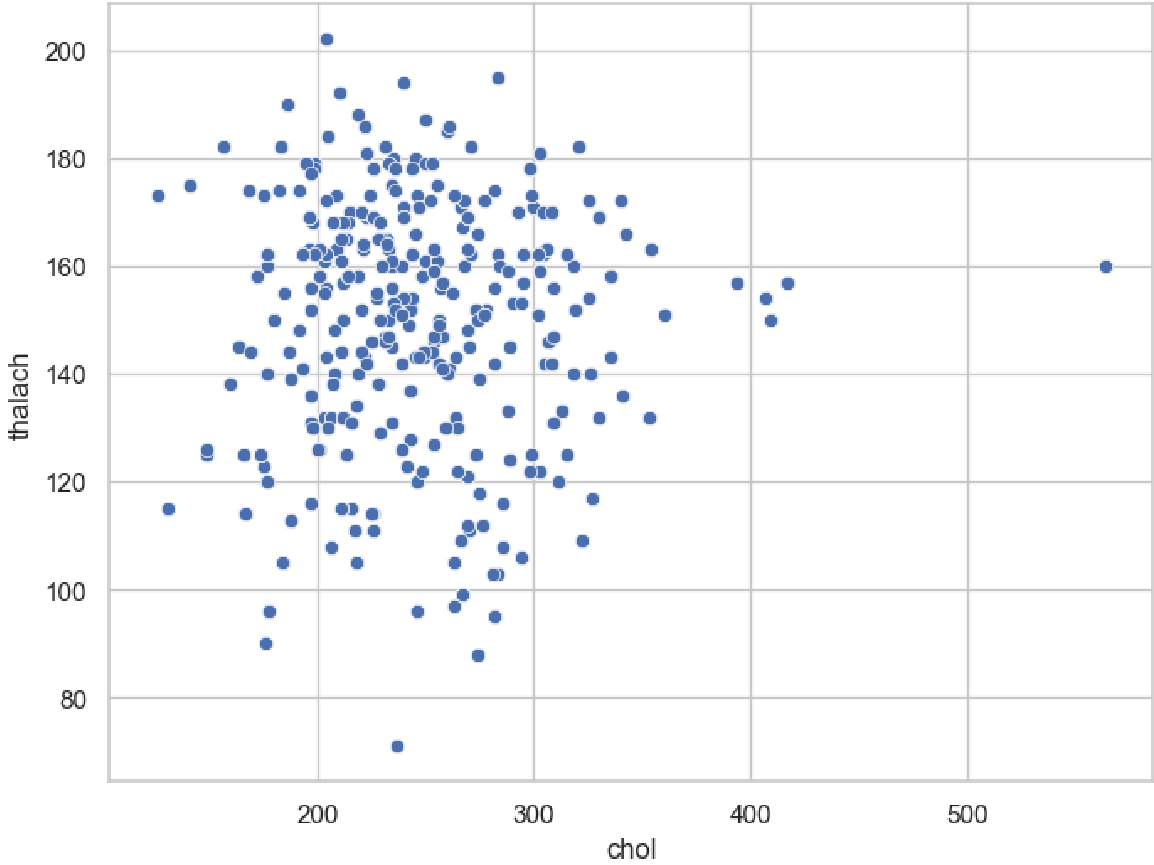


In [51]:
```
f, ax = plt.subplots(figsize=(8, 6))
ax = sns.regplot(x="age", y="chol", data=df)
plt.show()
```

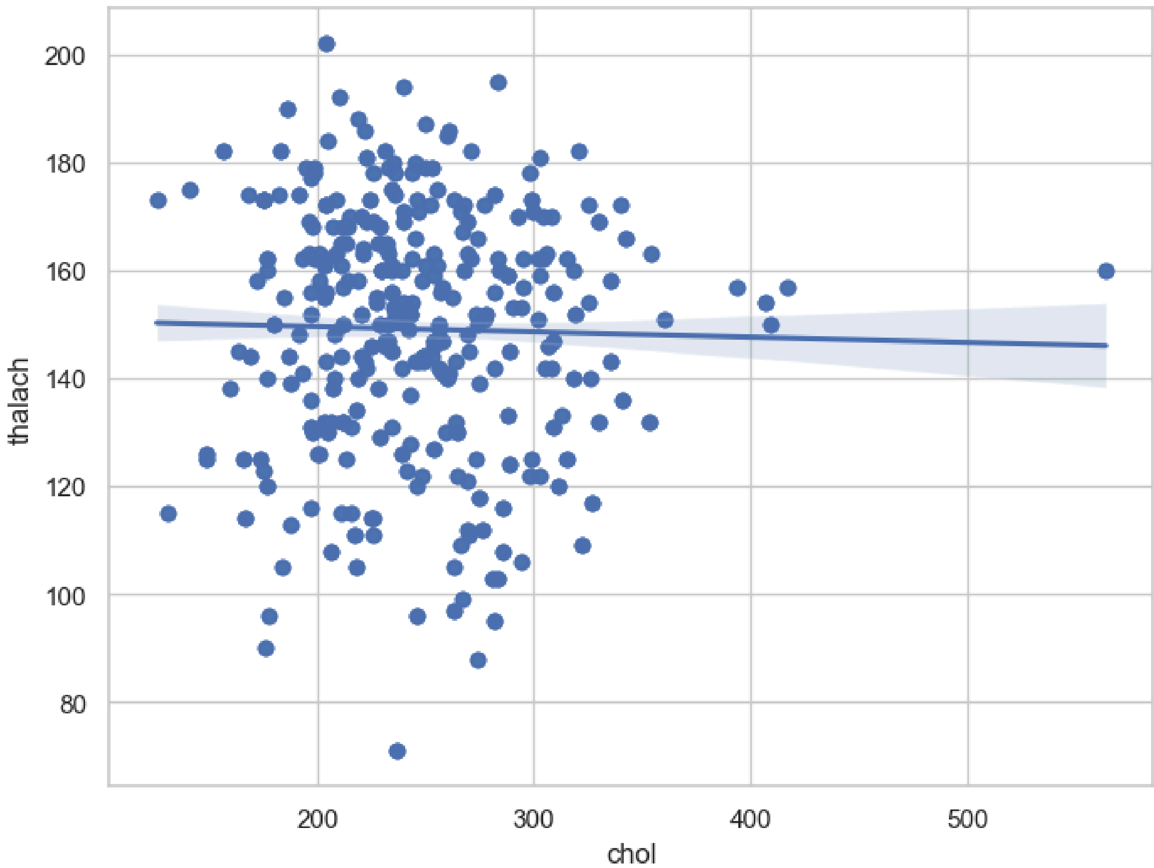


In [52]:
```
f, ax = plt.subplots(figsize=(8, 6))
ax = sns.scatterplot(x="chol", y = "thalach", data=df)
```

```
plt.show()
```



In [53]:
```
f, ax = plt.subplots(figsize=(8, 6))
ax = sns.regplot(x="chol", y="thalach", data=df)
plt.show()
```



Dealing with missing variables

In [54]:
```
df.isnull()
```

Out[54]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope
0	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...
1020	False	False	False	False	False	False	False	False	False	False	False
1021	False	False	False	False	False	False	False	False	False	False	False
1022	False	False	False	False	False	False	False	False	False	False	False
1023	False	False	False	False	False	False	False	False	False	False	False
1024	False	False	False	False	False	False	False	False	False	False	False

1025 rows × 14 columns

In [56]:
```
df.isnull().any().any()
```

Out[56]: False

In [57]:
```
df.isnull().values.any()
```

Out[57]: False

In [58]:
```
df.isnull().values.sum()
```

Out[58]: 0

In [59]:
```
df.isnull().sum()
```

Out[59]:
```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

Checking with Assert statement

In [ ]:
```
assert pd.notnull(df).all().all() # assert will not return anything if the condi
```

In [62]:
```
assert(df>=0).all().all()
```

In [63]:
```
assert(df<=0).all().all()
```

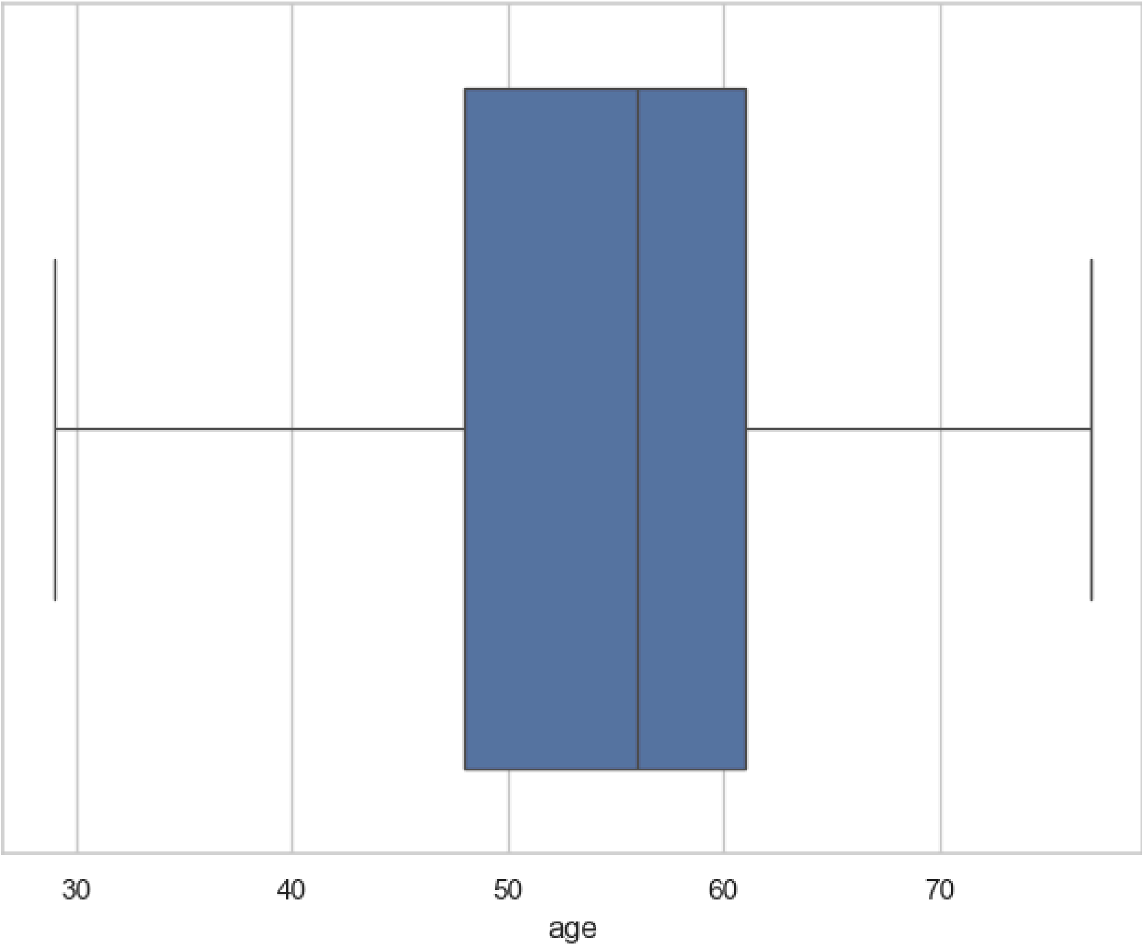
```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[63], line 1
----> 1 assert(df<=0).all().all()

AssertionError:
```

Outlier Detection

In [64]:
```
df['age'].describe()
```

Out[64]:
```
count    1025.000000
mean       54.434146
std         9.072290
min        29.000000
25%        48.000000
50%        56.000000
75%        61.000000
max        77.000000
Name: age, dtype: float64
```

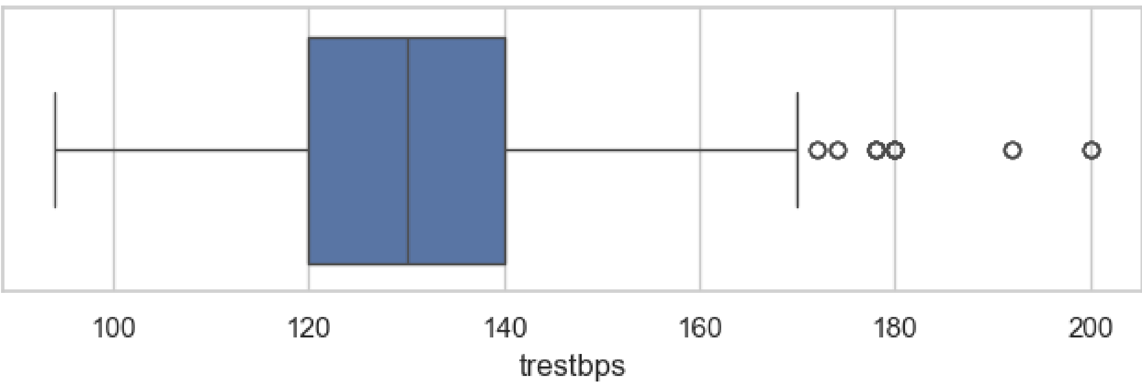
In [66]:
```
f,ax=plt.subplots(figsize=(8,6))
ax = sns.boxplot(x=df['age'])
plt.show()
```



In [67]:
```
df['trestbps'].describe()
```

Out[67]:
```
count    1025.000000
mean      131.611707
std        17.516718
min        94.000000
25%       120.000000
50%       130.000000
75%       140.000000
max       200.000000
Name: trestbps, dtype: float64
```

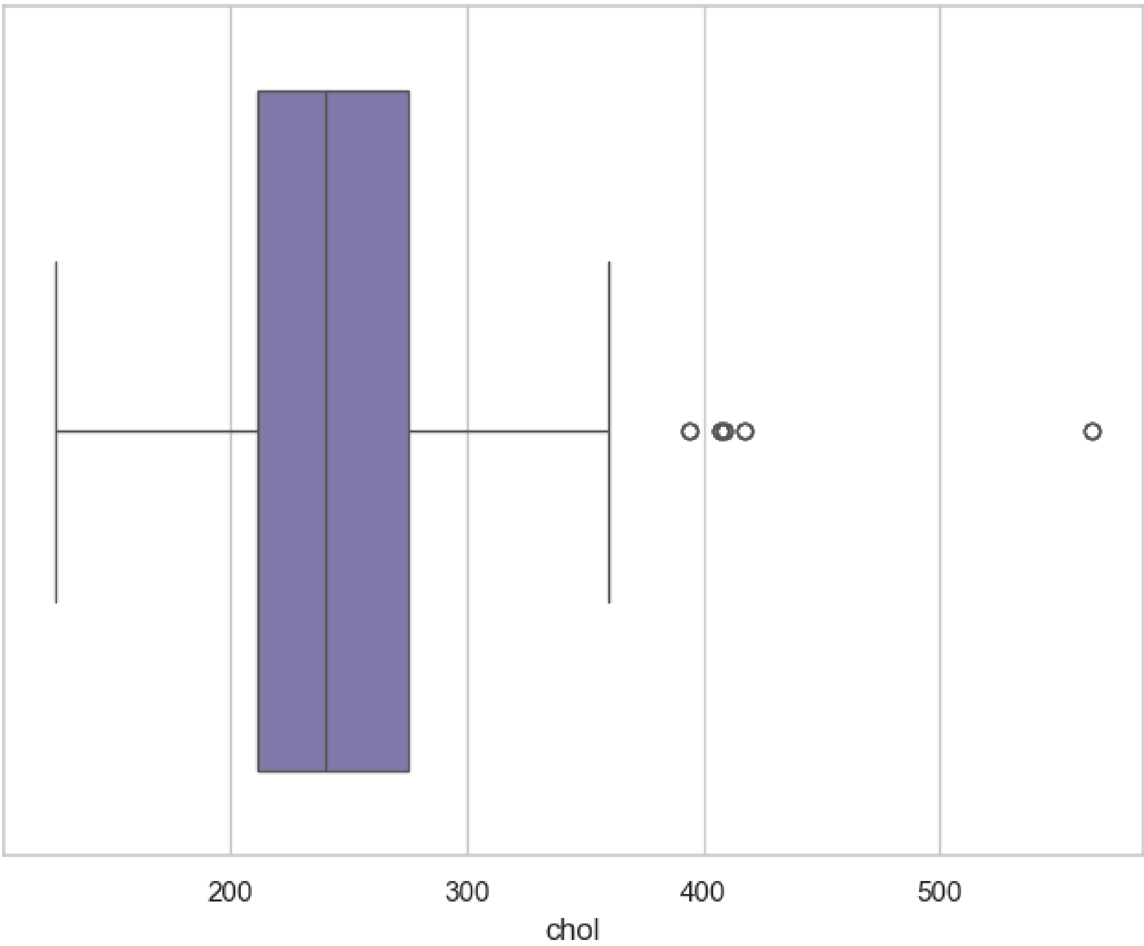
In [68]:
```
f,ax=plt.subplots(figsize=(8,2))
ax=sns.boxplot(x=df['trestbps'])
plt.show(

)
```



In [70]:
```
df['chol'].describe(

)
```

Out[70]:
```
count    1025.00000
mean      246.00000
std        51.59251
min       126.00000
25%       211.00000
50%       240.00000
75%       275.00000
max       564.00000
Name: chol, dtype: float64
```

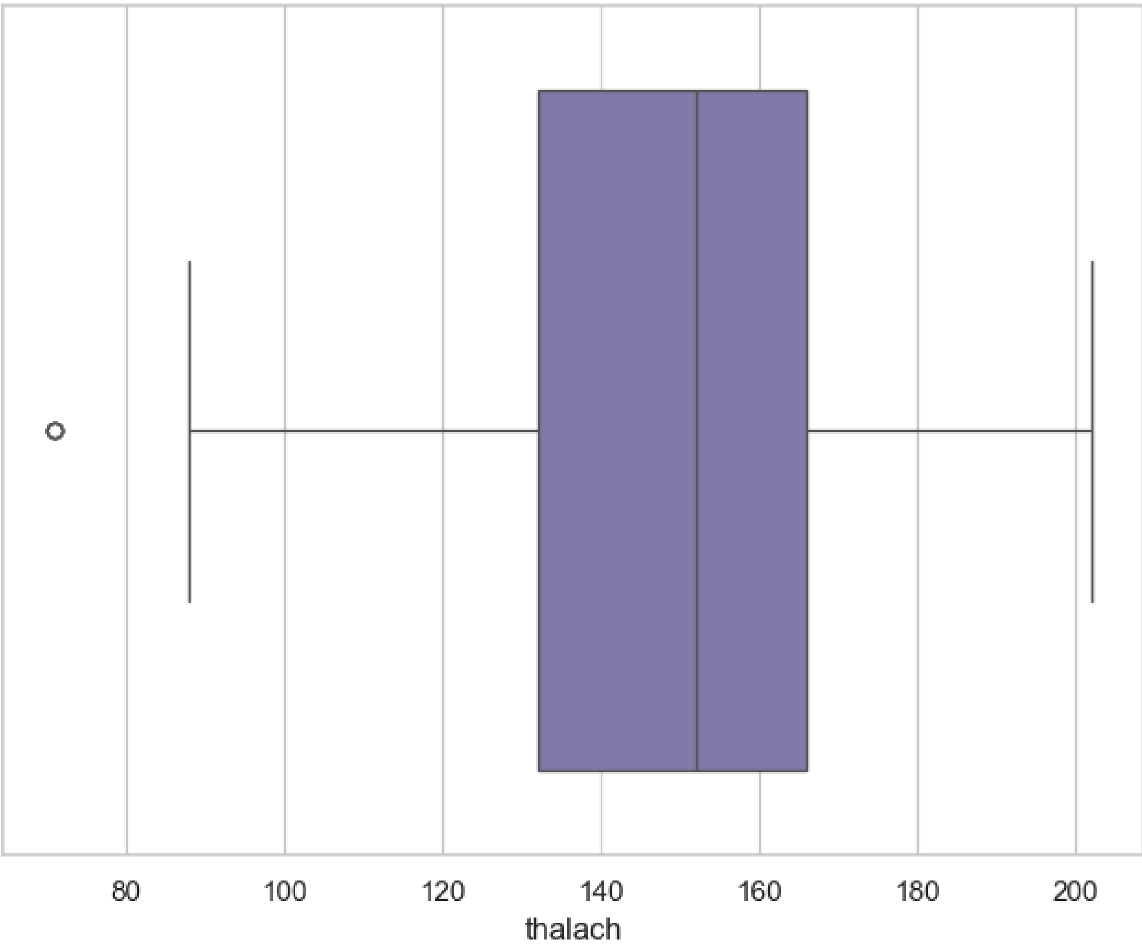
In [72]:
```
f,ax=plt.subplots(figsize=(8,6))
ax=sns.boxplot(x=df['chol'], color='m')
plt.show(

)
```



In [73]:
```
df['thalach'].describe()
```

Out[73]:
```
count    1025.000000
mean      149.114146
std        23.005724
min        71.000000
25%       132.000000
50%       152.000000
75%       166.000000
max       202.000000
Name: thalach, dtype: float64
```

In [76]:
```
f,ax=plt.subplots(figsize=(8,6))
ax=sns.boxplot(x=df['thalach'], color='m')
plt.show(

)
```



In [77]:
```
df['oldpeak'].describe()
```

Out[77]:
```
count    1025.000000
mean        1.071512
std         1.175053
min         0.000000
25%         0.000000
50%         0.800000
75%         1.800000
max         6.200000
Name: oldpeak, dtype: float64
```

In [78]:
```
f,ax=plt.subplots(figsize=(8,6))
ax=sns.boxplot(x=df['oldpeak'])
```

```
plt.show(

)
```